

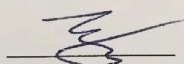
МИНОБРАЗОВАНИЯ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

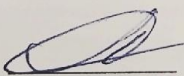
Факультет компьютерных наук
Кафедра технологий обработки и защиты информации

*Разработка многокомпонентной системы двунаправленного перевода
русского жестового языка с применением методов глубокого обучения*

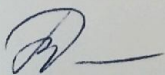
Бакалаврская работа
09.03.02 Информационные системы и технологии
Профиль «Обработка информации и машинное обучение»

Зав. кафедрой  Сирота А. А. д.т.н., профессор 18.06.2026

Обучающийся  Черных А. В.

Руководитель  Дрюченко М. А. к.т.н., доцент

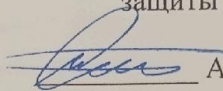
Воронеж 2026



МИНОБРАЗОВАНИЯ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра технологий обработки и защиты информации

УТВЕРЖДАЮ
заведующий кафедрой
технологий обработки и
защиты информации

 А. А. Сирота
_____._____.2026

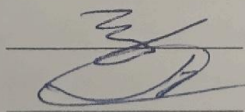
ЗАДАНИЕ
НА ВЫПОЛНЕНИЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
ОБУЧАЮЩЕГОСЯ Черных Александра Владимировича

1. Тема работы «Разработка многокомпонентной системы двунаправленного перевода русского жестового языка с применением методов глубокого обучения», утверждена решением ученого совета факультета компьютерных наук от 08.10.2025
2. Направление подготовки: 09.03.02 Информационные системы и технологии
3. Срок сдачи законченной работы: 10.06.2026
4. Календарный план (строится в соответствии со структурой ВКР):

№	Структура ВКР	Сроки выполнения	Примечание
1	Введение	01.09.2025 – 15.10.2025	
2	1. Анализ предметной области	01.09.2025 – 01.11.2025	
3	1.1 Русский жестовый язык и задача машинного перевода	01.09.2025 – 15.10.2025	
4	1.2 Обзор существующих методов и систем	15.10.2025 – 01.11.2025	
5	1.3 Постановка задачи и требования к системе	01.11.2025 – 15.11.2025	
6	2. Архитектура и проектирование системы	15.11.2025 – 01.02.2026	
7	2.1 Общая архитектура микросервисной системы	15.11.2025 – 15.12.2025	
8	2.2 Методы распознавания жестов	15.12.2025 – 15.01.2026	
9	2.3 Языковые модели, синтез речи и RAG	15.01.2026 – 01.02.2026	
10	3. Реализация и экспериментальное исследование	01.02.2026 – 01.05.2026	
11	3.1 Реализация компонентов системы	01.02.2026 – 01.03.2026	
12	3.2 Предобработка данных и обучение моделей	01.03.2026 – 01.04.2026	
13	3.3 Экспериментальное обоснование архитектурных решений	01.04.2026 – 01.05.2026	
14	Заключение	01.05.2026 – 15.05.2026	
15	Список использованных источников	15.05.2026 – 25.05.2026	
16	Приложение	25.05.2026 – 01.06.2026	

Обучающийся

Руководитель



Черных А. В.

Дрюченко М. А.

РЕФЕРАТ

Бакалаврская работа 60 с., 11 рис., 3 табл., 38 использованных источников, 1 приложение.

РУССКИЙ ЖЕСТОВЫЙ ЯЗЫК, РАСПОЗНАВАНИЕ ЖЕСТОВ, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, НЕЙРОННЫЕ СЕТИ, МИКРОСЕРВИСНАЯ АРХИТЕКТУРА, ЯЗЫКОВЫЕ МОДЕЛИ, СИНТЕЗ РЕЧИ, МАШИННЫЙ ПЕРЕВОД, MLOPS, ДОПОЛНЕННАЯ ГЕНЕРАЦИЯ С ИЗВЛЕЧЕНИЕМ

Объект исследования – методы и алгоритмы автоматического распознавания и перевода русского жестового языка на основе глубокого обучения.

Предмет исследования – многокомпонентная программная система двунаправленного перевода РЖЯ, функционирующая в режиме реального времени.

Цель исследования – разработка многокомпонентной программной системы, обеспечивающей двунаправленный перевод между русским жестовым языком и устной русской речью в режиме реального времени с задержкой не более 2000 мс.

Результаты работы: спроектирована и реализована микросервисная система из пяти компонентов; разработан DVC-пайплайн предобработки корпуса Slovo с многопоточным извлечением ключевых точек DWPose; обучена и сравнена серия из четырех вариантов модели классификации жестов на основе ST-GCN; интегрирована языковая модель Qwen2-1.5B-Instruct с дообучением LoRA и опциональным механизмом RAG на базе Qdrant; проведено девять экспериментов по обоснованию архитектурных решений с трекингом в MLflow.

Область применения результатов: системы доступности для людей с нарушениями слуха и речи, государственные и муниципальные сервисы, медицинские учреждения, банковские отделения обслуживания клиентов.

СОДЕРЖАНИЕ

РЕФЕРАТ	2
СОДЕРЖАНИЕ.....	4
ВВЕДЕНИЕ.....	5
1 Анализ предметной области.....	9
1.1 Русский жестовый язык: лингвистические основы и особенности.....	9
1.2 Корпус Slovo и особенности данных	10
1.3 Обзор существующих методов распознавания жестов	12
1.4 Обзор систем машинного перевода жестового языка	17
1.5 Постановка задачи и требования к системе	19
2 Архитектура и проектирование системы.....	21
2.1 Общая архитектура микросервисной системы	21
2.2 Методы извлечения ключевых точек и классификации жестов.....	23
2.3 Автоматическое распознавание речи	26
2.4 Перевод глосс с применением языковых моделей и RAG.....	27
2.5 Синтез речи.....	29
2.6 MLOps-инфраструктура	31
3 Реализация и экспериментальное исследование.....	34
3.1 Реализация компонентов системы.....	34
3.2 Предобработка датасета Slovo и обучение классификатора.....	35
3.3 Экспериментальное обоснование архитектурных решений.....	42
ЗАКЛЮЧЕНИЕ	50
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	53
ПРИЛОЖЕНИЕ А	58

ВВЕДЕНИЕ

Обеспечение доступности информационных и коммуникационных технологий для людей с ограниченными возможностями здоровья является одной из приоритетных задач современного общества. Согласно данным Всемирной организации здравоохранения, по состоянию на 2023 год более 430 миллионов человек в мире страдают от нарушений слуха, требующих медицинской или реабилитационной помощи. Из них около 70 миллионов человек используют жестовые языки как основное или единственное средство коммуникации. На территории Российской Федерации число граждан с нарушениями слуха превышает 13 миллионов человек; для значительной части из них русский жестовый язык (РЖЯ) является родным и единственным привычным средством общения.

В 2013 году федеральным законом «Об образовании в Российской Федерации» РЖЯ был признан языком общения глухих, что создало правовую основу для развития соответствующих языковых технологий и инструментов поддержки. Несмотря на это, практические системы автоматического перевода РЖЯ, доступные широкому кругу пользователей, по-прежнему остаются редкостью. Существующие коммерческие решения, такие как мобильное приложение «Сурдофон» и веб-сервис «Сурдосервер», характеризуются ограниченным словарем (несколько сотен жестов), высокими требованиями к качеству видеосъемки и отсутствием двунаправленного перевода.

Академические системы, публикуемые в рамках научных исследований, как правило, ориентированы на американский (ASL) или британский (BSL) жестовые языки и требуют специализированного сенсорного оборудования — перчаток с датчиками или глубинных камер (Microsoft Kinect, Intel RealSense).

Таким образом, в настоящее время в России существует значительный разрыв между потребностью в системах перевода РЖЯ и доступными технологическими решениями.

Разработка открытой, точной и масштабируемой системы, работающей на стандартном потребительском оборудовании с обычной веб-камерой и без специализированных датчиков, представляет собой актуальную научно-практическую задачу.

Актуальность данной работы обусловлена совпадением ряда факторов:

- появлением открытого корпуса русского жестового языка Slovo (SberDevices, 2023), содержащего 20 400 видеозаписей 1000 жестов РЖЯ и представляющего первый крупный открытый датасет для данной задачи;
- достижением современными методами скелетного распознавания жестов (DWPose + графовые нейронные сети) точности, сопоставимой с методами на основе необработанного видео, при значительно меньших вычислительных затратах;
- появлением открытых больших языковых моделей (LLM) с поддержкой русского языка, пригодных для точного перевода последовательностей глосс в естественный текст;
- развитием эффективных методов дообучения LLM (LoRA, QLoRA), позволяющих адаптировать языковые модели под специфику жестового языка без использования дорогостоящей вычислительной инфраструктуры;
- распространением контейнерных технологий, обеспечивающих воспроизводимое развертывание сложных многокомпонентных систем.

Целью настоящей выпускной квалификационной работы является разработка и экспериментальное исследование многокомпонентной программной системы Glossa, обеспечивающей автоматический двунаправленный перевод русского жестового языка в режиме реального времени с задержкой, не превышающей 2 секунды на мобильном устройстве POCO M5 в 4G-сети.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) провести анализ методов распознавания жестового языка и машинного перевода, сформулировать требования к разрабатываемой системе;
- 2) спроектировать микросервисную архитектуру системы, определить интерфейсы взаимодействия компонентов и схему передачи данных;
- 3) разработать модуль компьютерного зрения для извлечения скелетных ключевых точек из видеопотока с применением DWPose (YOLOX-детектор + RTMPose);
- 4) разработать и обучить модель классификации жестов на основе пространственно-временной графовой сверточной сети (ST-GCN) на корпусе Slovo;
- 5) реализовать алгоритм скользящего окна для непрерывного распознавания жестов в видеопотоке;
- 6) интегрировать языковую модель Qwen2-1.5B-Instruct с методом дообучения LoRA и механизмом дополненной генерации с извлечением (RAG) для перевода последовательностей глосс в русские предложения;
- 7) реализовать компонент автоматического распознавания речи на основе модели Whisper и компонент синтеза речи на основе Silero TTS;
- 8) провести серию из девяти экспериментов для обоснования ключевых архитектурных решений системы с трекингом результатов в MLflow;
- 9) разработать MLOps-инфраструктуру: DVC-пайплайны версионирования данных и моделей, реестр моделей с SLO-гейтами, мониторинг в эксплуатации.

Объектом исследования являются методы и алгоритмы автоматической обработки данных жестового языка.

Предметом исследования является многокомпонентная программная система двунаправленного перевода РЖЯ, функционирующая в режиме реального времени.

Методами исследования являются: методы глубокого обучения [1, 2] (графовые нейронные сети, трансформеры, диффузионные модели); методы компьютерного зрения (извлечение ключевых точек, нормализация скелетных данных); методы оценки качества машинного перевода (BLEU, ROUGE); статистические методы оценки (кросс-валидация, доверительные интервалы, оценка калибровки).

Практическая значимость работы состоит в создании открытого программного комплекса, пригодного для развертывания в учреждениях, обслуживающих граждан с нарушениями слуха, а также в разработке воспроизводимой MLOps-инфраструктуры, позволяющей продолжить улучшение системы по мере накопления данных.

1 Анализ предметной области

1.1 Русский жестовый язык: лингвистические основы и особенности

Русский жестовый язык представляет собой самостоятельную естественную лингвистическую систему, сложившуюся в среде глухих граждан России. Подобно всем жестовым языкам мира, РЖЯ обладает собственной грамматикой, лексикой, синтаксисом и фонологией, независимыми от соответствующих уровней русского языка. Ошибочное отождествление жестового языка с «жестикულიрованным русским» (калькированной жестовой речью, КЖР) является распространенным заблуждением: КЖР представляет собой искусственную систему, создававшуюся для визуального сопровождения устной речи, тогда как РЖЯ — полноценный язык с многовековой историей.

С точки зрения лингвистики, РЖЯ использует трехмерное пространство перед говорящим (пространство жестикуляции) в качестве грамматического инструмента. Местоположение и направление движения руки в пространстве кодирует грамматические отношения: субъект, объект, направление действия, время. Форма руки (конфигурация пальцев) является аналогом фонемы в устных языках: одна конфигурация, измененная в другую, меняет смысл знака так же, как замена одного звука на другой меняет слово. Помимо мануальных компонентов (движений рук), РЖЯ активно использует немануальные компоненты: мимику лица, движения корпуса, направление взгляда. Например, вопросительное предложение маркируется специфическим выражением лица (поднятые брови и наклон головы для общевпросительных, сдвинутые брови для частновпросительных), а не интонацией.

Базовой единицей РЖЯ является знак (жест), однозначно соответствующий лексической единице. Словарь РЖЯ насчитывает несколько тысяч знаков; в Академическом словаре РЖЯ зафиксировано более 3000 единиц. Грамматические отношения, не имеющие лексического выражения, кодируются пространственными операциями над знаками.

Для нужд машинного перевода жестового языка широко применяется нотация глосс — условная запись знаков жестового языка словами языка большинства в словарной форме (леммой). Глосс является меткой знака, но не его переводом: знак МАМА в РЖЯ не тождественен слову «мама» в русском языке, хотя именно им и помечается в глоссовой записи. Применение глоссовой нотации позволяет разбить задачу перевода жестового языка на два этапа: распознавание последовательности знаков и формирование из нее текста на русском языке. Такое разделение повышает интерпретируемость системы и упрощает внесение словарных и грамматических знаний через механизм RAG.

Особенностью РЖЯ, важной для задачи распознавания, является ручная дактилология — побуквенное воспроизведение слов с помощью дактильного алфавита (33 жеста, соответствующих буквам русского алфавита). Дактиль применяется для написания имен собственных, технических терминов и заимствованных слов, не имеющих собственного знака в РЖЯ. Корректное распознавание дактиля требует различения тонких конфигурационных различий между похожими буквами (например, Б, В, Г, Д), что является одной из наиболее сложных подзадач.

1.2 Корпус Slovo и особенности данных

В качестве основного обучающего корпуса для разработки классификатора жестов в настоящей работе используется датасет Slovo, опубликованный компанией SberDevices в 2023 году. Датасет представляет собой первый крупный открытый корпус русского жестового языка и является в настоящее время наиболее полным общедоступным ресурсом в данной области.

Датасет Slovo содержит 20 400 видеозаписей, охватывающих 1000 знаков РЖЯ плюс специальный класс no_event (видеозаписи без жестов). По 15 записей на класс отнесены к обучающей выборке, по 5 — к проверочной. Записи выполнены 194 дикторами различного возраста, пола и места проживания, что обеспечивает вариабельность и снижает переобучение на

идиосинкразии отдельных исполнителей. Видеозаписи выполнены в бытовых условиях с разнообразными фонами и условиями освещения; разрешение видео составляет 640×480 или 1280×720 пикселей при частоте 25 кадров в секунду.

По результатам разведочного анализа данных (EDA), проведенного в рамках эксперимента 00 настоящей работы, установлены следующие характеристики корпуса:

- общее число записей (без `no_event`): 20 400, из которых 15 300 обучающих и 5100 проверочных;
- средняя длина жеста (по аннотациям `begin/end`): 50,7 кадра, медиана — 48 кадров, стандартное отклонение — 21,4 кадра; 75-й перцентиль составляет 58 кадров, что обосновывает выбор длины последовательности $T = 64$ кадра как покрывающего ~80% записей без необходимости агрессивного обрезания;
- дисбаланс классов достигает 20-кратного значения: наиболее частый класс содержит в 20 раз больше записей, чем наименее частый; класс `no_event` удален из обучающей выборки как нецелевой и вносящий значительный дисбаланс;
- полнота ключевых точек, извлекаемых `DWPose`: для 87,3% кадров оба запястья определяются корректно, для 12,7% кадров одна или обе кисти не определяются (отсутствуют в поле зрения или перекрыты); нулевые векторы используются в качестве заполнителей для неопределенных точек.

Аннотации датасета хранятся в файле `annotations.csv` со столбцами:

- `attachment_id` (идентификатор видеофайла), `text` (глосс), `user_id`, `height`, `width`;
- `length` (полная длина видео в кадрах), `train` (булев флаг обучающей выборки);
- `begin`, `end` (границы жеста в кадрах). Видеофайлы организованы по директориям;

- train/ и val/, имена файлов соответствуют attachment_id с расширением .mp4;

Для обеспечения воспроизводимости загрузки данных разработан класс SlovoDataset, поддерживающий несколько соглашений об именовании путей к файлам (в том числе структуру Kaggle Datasets) и корректно обрабатывающий оба варианта CSV-схемы — с булевым столбцом train и со строковым столбцом split.

Методология сбора корпуса Slovo основана на краудсорсинговом подходе с многоступенчатым контролем качества. Работы проводились на двух платформах: Яндекс.Толока (сбор видеозаписей) и ABC Elementary (валидация и разметка). На квалификационном этапе проверялось знание РЖЯ: из 45 тысяч человек, проявивших интерес, экзамен полностью завершили 25 тысяч, из которых около 400 получили оценку не ниже 90% и были допущены к стадиям контроля качества.

Каждая видеозапись проверялась тремя–пятью независимыми экспертами с динамическим увеличением числа проверяющих при несогласованных ответах.

Финальная разметка временных границ жеста для каждого видео выполнялась тремя специалистами с последующим алгоритмическим усреднением. Достигнутый уровень качества подтверждается тем фактом, что из более чем 100 тысяч проаннотированных записей агрегация разметки не удалась лишь в 6 случаях [3].

1.3 Обзор существующих методов распознавания жестов

Современные методы автоматического распознавания жестов (Sign Language Recognition, SLR) [4] подразделяются на два основных класса: методы на основе RGB-видео и методы на основе скелетных ключевых точек. Оба класса применяются как для изолированного распознавания (Isolated SLR — классификация отдельного знака из видеосегмента), так и для непрерывного распознавания (Continuous SLR — обнаружение и декодирование последовательности знаков из непрерывного потока).

Задача Continuous SLR неотделима от задачи временной сегментации — выделения границ отдельных знаков в непрерывном потоке. SubUNets [5] и архитектуры на основе CTC решают сегментацию и распознавание совместно через мягкое выравнивание (soft alignment) меток без явной разметки границ. Альтернативный класс методов выполняет сегментацию явно, как самостоятельный предшествующий этап: метод выравнивания последовательностей по динамическому программированию [6], предсказание будущих кадров для обнаружения смены сцены [7] и трансформерные архитектуры для посегментной классификации действий, такие как ASFormer [8] и временные сверточные сети (Temporal Convolutional Networks, TCN) [9]. Применительно к жестовому языку без сенсоров (акселерометров) Хань и Бессмертный [10] показали, что границу между словами можно надежно определить по динамике евклидова расстояния между координатами кисти, предплечья, глаз и рта, вычисленными детектором ключевых точек, с пороговым методом сегментации; алгоритм апробирован на 4364 видео вьетнамского жестового языка [11] и показал точность, сопоставимую с ручной разметкой, при низкой ресурсоемкости. Аналогичные пороговые и НММ-методы сегментации применяются и в задачах распознавания речи [12, 13]. В системе Glossa аналогичная задача решается алгоритмически — скользящим окном фиксированной длины с фильтрацией по порогу уверенности — без обучаемого сегментирующего блока; переход к явной сегментации границ знаков обозначен в заключении как направление дальнейшего развития.

Для языков с ограниченными ресурсами разметки (low-resource sign languages) существенным фактором является доступность крупных датасетов: WLASL для американского жестового языка [14], BosphorusSign для турецкого [15], а также работы по непрерывному распознаванию с выравниванием признаков (VAC) [16] и трансляции через распознавание ключевых точек тела [17]. Для вьетнамского жестового языка, в частности, отмечается

нерешенность базовых вопросов стандартизации словаря и аннотирования [18], что характерно и для русского жестового языка до появления корпуса Slovo [19].

Методы на основе RGB-видео. Трехмерные сверточные нейронные сети (3D CNN) обрабатывают видео как трехмерный тензор (время $\times$ высота $\times$ ширина), применяя свертки по всем трем измерениям одновременно. Архитектура I3D (Inflated 3D ConvNet), предложенная Carreira и Zisserman в 2017 году [20], получается раздуванием ядер 2D-сверток предобученной ImageNet сети до трехмерных, что обеспечивает инициализацию с Transfer Learning. Модель S3D (Separable 3D CNN) [21] разделяет трехмерные свертки на пространственные 2D и временные 1D, сокращая число параметров при сохранении точности. Двухпоточные архитектуры (Two-Stream Networks) обрабатывают пространственный поток (отдельные кадры) и временной поток (оптический поток) отдельно, после чего объединяют предсказания. Сеть SlowFast [22] использует медленный поток с высоким пространственным разрешением для захвата семантики сцены и быстрый поток с высокой частотой кадров для захвата мелких движений.

Модель S3D, примененная компанией SberDevices для работы с корпусом Slovo, достигает точности top-1 89,1% при 400 млн параметров. Основные недостатки RGB-методов применительно к задаче перевода РЖЯ:

- высокие вычислительные требования: для инференса S3D необходима видеопамять не менее 8 Гбайт; применение на CPU недопустимо из-за задержки >10 с на одну запись;
- чувствительность к фону и условиям освещения: изменение освещения или фона существенно снижает точность без дополнительной предобработки;
- ограниченная интерпретируемость промежуточных представлений;
- отсутствие естественного разбиения на именованные части тела, что затрудняет включение анатомических ограничений.

Методы на основе скелетных ключевых точек. Данный класс методов сначала извлекает координаты ключевых точек тела (суставов, кончиков пальцев, характерных точек лица), после чего обрабатывает временные последовательности этих координат. Ключевым компонентом является детектор поз: OpenPose, HRNet, BlazePose, OpenPose или RTMPose (лежащий в основе DWPose) [23–25].

Фреймворк DWPose [26], применяемый в настоящей работе, реализует двухэтапную схему «детектор + поза»: легкий детектор человека YOLOX [26] определяет ограничивающий прямоугольник, после чего модель RTMPose [27] оценивает 133 ключевые точки в формате COCO-WholeBody (17 точек тела, 6 точек стоп, 68 точек лица, по 21 точке на кисть) в пределах найденного прямоугольника. Для задачи распознавания РЖЯ полная 133-точечная разметка избыточна и не используется напрямую: лицевые точки (68) отбрасываются, а из оставшихся формируется итоговый вектор из 75 узлов — 17 точек тела COCO, 6 точек стоп и 10 дублирующих узлов опорных суставов (плечи, бедра, колени, лодыжки), дополняющих позу до 33 узлов, плюс по 21 точке на кисть. Каждый узел описывается тройкой (x, y, confidence), что дает вектор размерностью 225. Двухэтапная схема DWPose обеспечивает большую устойчивость распознавания кистей по сравнению с однократными решениями за счет отдельной модели RTMPose, специализированной на оценке поз, и достигает на CPU производительности порядка 15–20 кадров/с при разрешении 640×480, что достаточно для скользящего окна с шагом $S = 15$ кадров при частоте видеопотока 25 FPS.



Рисунок 1 – Конвейер DWPose.

Для обработки скелетных последовательностей применяются различные архитектуры. Рекуррентные сети (LSTM, GRU) обрабатывают последовательности ключевых точек как временные ряды. Однако они не учитывают явно пространственную структуру графа скелета и плохо масштабируются на длинные последовательности из-за проблемы затухающих градиентов.

Пространственно-временные графовые сверточные сети (Spatial-Temporal GCN, ST-GCN), предложенные Yan et al. в 2018 году [28], моделируют скелет как граф, где узлы соответствуют ключевым точкам, а ребра — анатомическим связям (суставам). Операция пространственной графовой свертки применяется к соседям каждого узла в соответствии с матрицей смежности, кодирующей анатомическую топологию. Временная свертка применяется вдоль временного измерения аналогично 1D-свертке. В работе применяется вариант ST-GCN с адаптивной матрицей смежности (Adaptive Graph Convolutional Network, AGCN) [29], в котором матрица смежности частично обучается совместно с остальными параметрами, что позволяет модели обнаруживать нелокальные зависимости, выходящие за рамки анатомически соседних суставов.

Преимущества скелетного подхода перед RGB-видео:

- требования к памяти на 10–15 раз меньше, чем у видеометодов; возможен инференс на CPU;
- инвариантность к фону и условиям освещения: ключевые точки нормируются относительно позы тела, а не абсолютных координат кадра;
- встроенная интерпретируемость: каждый узел графа соответствует конкретной части тела;
- естественная возможность применения аугментаций, учитывающих анатомию: горизонтальное отражение со сменой ролей левой и правой руки.

Недостатком является чувствительность качества к точности детектора поз: при перекрытии рук, нестандартных ракурсах или частичном выходе кистей из поля зрения точность снижается. В настоящей работе данная проблема частично решается заполнением ненаблюдаемых точек нулевыми векторами и применением аугментации с добавлением случайного шума к ключевым точкам.

Принципиально важным для практического применения является вопрос ускорения инференса скелетных моделей на CPU без специализированных ускорителей. Опыт команды Sber AI показал, что последовательное применение оптимизаций дает кратный прирост производительности: конвертация исходной PyTorch-модели в формат ONNX (Open Neural Network Exchange) ускоряет инференс на CPU приблизительно в 10–12 раз; дополнительная активация OpenVINO Execution Provider в составе ONNX Runtime обеспечивает еще 1,5-кратное ускорение и позволяет распознавать от 3 до 3,5 жестов в секунду на типовом ноутбуке без видеокарты [30]. Таким образом, скелетный подход в сочетании с цепочкой ONNX + OpenVINO делает возможным развертывание систем распознавания жестового языка на широком классе потребительского оборудования.

1.4 Обзор систем машинного перевода жестового языка

Задача перевода жестового языка (Sign Language Translation, SLT) — получение текста на естественном языке из видеозаписи — представляет собой задачу более высокого уровня по сравнению с изолированным распознаванием. В отличие от SLR, SLT требует не только правильно распознать отдельные знаки, но и восстановить грамматическую структуру высказывания, которая в жестовом языке может существенно отличаться от структуры эквивалентного устного предложения.

Архитектура Sign Language Transformers, предложенная Camgoz et al. в 2020 году [31], выполняет совместное обучение распознавания (СТС) и перевода (seq2seq) в едином трансформере, разделяя кодировщик для

последовательности ключевых точек и декодировщик для генерации текста. На датасете PHOENIX-2014T (немецкий жестовый язык) данная архитектура достигла BLEU-4 = 0,219. Более поздние работы на датасете How2Sign (американский жестовый язык) с применением больших языковых моделей достигают BLEU-4 в диапазоне 0,30–0,45.

Для русского жестового языка публично доступные системы перевода практически отсутствуют. Наиболее близким аналогом является работа SberDevices по оценке S3D на корпусе Slovo, однако она ограничивается задачей классификации и не включает компонент перевода в текст.

Проблема нехватки профессиональных сурдопереводчиков носит системный характер. По оценкам Всероссийского общества глухих, дефицит квалифицированных специалистов в данной области превышает четыре тысячи человек. Сложность подготовки кадров обусловлена уникальностью каждого национального жестового языка, наличием региональных диалектов и тем, что основная часть носителей РЖЯ — слышащие дети глухих родителей — составляет крайне небольшую группу. Согласно прогнозам Всемирной организации здравоохранения, к 2050 году потерей слуха в той или иной мере будет страдать каждый десятый житель планеты — более 700 миллионов человек, что многократно усиливает социальную значимость автоматических систем перевода жестового языка [30].

Коммерческие системы для РЖЯ:

- «Сурдофон» (Россия) — мобильное приложение для перевода речи в жесты и обратно; словарь ограничен ~300 жестами; не предоставляет открытого API;
- «Сурдосервер» — веб-сервис с видеотолкователями; перевод выполняется операторами-людьми, а не автоматически;
- международные коммерческие системы (SignAll, Signapse) ориентированы на ASL и BSL, не поддерживают РЖЯ.

Открытых программных комплексов для двунаправленного перевода РЖЯ, охватывающих весь конвейер от видеопотока до синтеза речи и обратно, в открытом доступе не существует. Настоящая работа направлена на заполнение данного пробела.

1.5 Постановка задачи и требования к системе

На основе проведенного анализа сформулированы требования к разрабатываемой системе Glossa.

Функциональные требования:

- реализация направления РЖЯ → речь: система принимает непрерывный видеопоток с веб-камеры, распознает жесты, переводит последовательность глосс в русское предложение и синтезирует аудио;
- реализация направления речь → РЖЯ: система принимает аудиовход, распознает речь, переводит текст в последовательность глосс для отображения;
- словарь не менее 1000 жестов;
- работа без специализированного оборудования: только стандартная веб-камера и микрофон;
- WebSocket-интерфейс для потоковой передачи видеок кадров и возврата результатов в режиме реального времени;
- поддержка базовых доменных словарей без переобучения классификатора жестов.

Нефункциональные требования (Service Level Objectives, SLO):

- сквозная задержка (E2E P95 latency) не более 2000 мс на устройстве POCO M5 (4G-сеть, RTT 65 мс, jitter 15 мс);
- точность классификатора жестов (top-1 accuracy) не менее 0,90;
- взвешенная метрика F1 классификатора жестов не менее 0,88;
- P95-задержка компонента распознавания жестов (CV + классификация) не более 50 мс на CPU;

- метрика WER (Word Error Rate) модели ASR не более 0,15;
- метрика BLEU-4 модели перевода глосс не менее 0,35;
- P95-задержка компонента NLP не более 500 мс.

2 Архитектура и проектирование системы

2.1 Общая архитектура микросервисной системы

Система Glossa спроектирована как совокупность пяти независимых микросервисов, взаимодействующих по четко определенным интерфейсам. Такой подход обусловлен рядом соображений: компоненты системы имеют существенно различные временные характеристики (от десятых долей миллисекунды для Redis до нескольких сотен миллисекунд для NLP), различные требования к ресурсам (объемная LLM-модель в NLP Service против легких CPU-моделей в остальных компонентах) и различные lifecycle — каждый компонент можно обновлять независимо, не затрагивая остальные.

Компоненты системы и их функции:

1) API Gateway (порт 8000) — центральный компонент, принимающий все входящие соединения. Реализован на FastAPI с асинхронной обработкой запросов. Поддерживает WebSocket (`/api/v1/ws/translate/{mode}`) для потоковой передачи видеокадров и аудиочанков от клиента и возврата результатов, а также REST эндпоинт `/api/v1/translate` для синхронного перевода. Выполняет маршрутизацию запросов к внутренним сервисам через HTTP-клиент (`httpx`), агрегацию ответов и управление WebSocket-сессиями. Каждой сессии присваивается уникальный идентификатор `session_id`, используемый для корреляции данных между компонентами и хранимый в Redis с TTL 5 минут.

2) CV Service (порт 8001) — сервис компьютерного зрения и классификации жестов. Принимает видеокадры (JPEG, base64) через HTTP POST `/process_frame` или WebSocket `/ws`. Применяет DWPose (YOLOX-детектор + RTMPose) для извлечения 75 ключевых точек скелета, нормализует их относительно центра бедер и масштабирует по ширине плеч. Формирует скользящий буфер кадров и в том же процессе выполняет инференс ST-GCN классификатора через ONNX Runtime (с опциональным OpenVINO Execution Provider), публикуя результаты классификации в Redis Stream «cv:results». В отличие от первоначального проектного решения о выделении классификации

жестов в отдельный микросервис, инференс ST-GCN был объединен с извлечением ключевых точек в одном процессе CV Service: оба этапа выполняются на CPU без GPU, имеют сопоставимую длительность (десятки миллисекунд) и совместно используют один и тот же входной кадр, поэтому межсервисный сетевой вызов добавлял бы задержку без выигрыша в масштабируемости.

3) ASR Service (порт 8002) — сервис автоматического распознавания речи. Принимает аудио (base64 PCM) через WebSocket. Использует faster-whisper с моделью Whisper base, int8-квантованием для CPU-инференса. Возвращает JSON с распознанным текстом, частичными (partial) сегментами и итоговой транскрипцией.

4) NLP Service (порт 8003) — сервис перевода глосс ↔ русский текст. Принимает последовательности глосс или русский текст через HTTP POST. Применяет языковую модель Qwen2-1.5B-Instruct с дообучением LoRA. Поддерживает опциональный механизм RAG для доменной адаптации лексики в базовой production-конфигурации RAG отключен (RAG_ENABLED=false) и включается явно при наличии развернутого хранилища эмбеддингов.

5) TTS Service (порт 8004) — сервис синтеза речи. Принимает текст через HTTP POST /synthesize. Использует Silero TTS v4 (русский язык). Возвращает аудио в формате WAV (24 кГц, mono). Кэширует часто используемые фразы в Redis для снижения задержки повторных запросов.

Обмен данными между компонентами осуществляется по двум механизмам. Синхронный обмен (HTTP REST через httpx.AsyncClient) применяется для операций, требующих немедленного ответа: вызовы API Gateway → CV Service, → ASR Service, → NLP Service, → TTS Service в рамках обработки одного запроса пользователя. Асинхронный обмен (Redis Streams) применяется для публикации промежуточных результатов конвейера (cv:results, asr:results, nlp:results) независимым потребителям — например, для

мониторинга или будущих сервисов аналитики — без изменения существующих продюсеров.

Совместная библиотека `glossa_common` содержит общие схемы данных (Pydantic-модели), утилиты логирования, трейсинга и телеметрии, используемые всеми сервисами. Это обеспечивает единообразие форматов API и упрощает интеграцию.

Инфраструктура мониторинга включает Prometheus (сбор метрик со всех сервисов по протоколу `/metrics`) и Grafana (дашборды для наблюдения за производительностью в реальном времени). Все сервисы экспортируют стандартные метрики: число запросов, задержки по перцентилям, частоту ошибок, лаг очередей Redis Streams. Распределенный трейсинг (OpenTelemetry/Jaeger) не входит в текущую production-конфигурацию и обозначен как направление дальнейшего развития MLOps-инфраструктуры.

2.2 Методы извлечения ключевых точек и классификации жестов

Компонент распознавания жестов решает задачу в два этапа: извлечение пространственных ключевых точек из каждого видеокadra и последующая классификация временных последовательностей ключевых точек.

Извлечение ключевых точек. Конвейер DWPose состоит из двух ONNX-моделей, исполняемых через `onnxruntime`: детектора человека YOLOX (вход 640×640 , постобработка — декодирование сетки предсказаний и немаксимальное подавление, NMS) и модели позы RTMPose (top-down, вход 288×384 , найденный прямоугольник обрезается аффинным преобразованием и подается на вход; выходные SimCC-представления координат по осям x и y декодируются через `argmax`). В отличие от MediaPipe, не использующего отдельного детектора и встроенного трекинга, текущая реализация выполняет полный проход «детекция + поза» для каждого кадра, что увеличивает вычислительные затраты, но снимает зависимость от качества межкадрового трекинга при быстрых движениях рук, характерных для жестовой речи.

Для каждого кадра формируется вектор из $75 \text{ точек} \times 3 \text{ координаты} = 225$

начений. Нормализация выполняется в два шага: вычитание координат центра бедер (среднее точек 11 и 12) обеспечивает трансляционную инвариантность; деление на расстояние между плечами (точки 5 и 6) обеспечивает масштабную инвариантность. После нормализации жест описывается относительным расположением конечностей, а не абсолютным положением в кадре, что снижает влияние различий в комплекции дикторов и расстоянии до камеры.

Формирование последовательностей. Из нормализованных ключевых точек формируются временные последовательности фиксированной длины $T = 64$ кадра.

Для записей короче 64 кадров применяется линейная интерполяция или дополнение нулями; для более длинных — равномерная дискретизация с фиксированным шагом.

Архитектура ST-GCN. Граф скелета $G = (V, E)$ определяется следующим образом: множество узлов $V = \{v_1, \dots, v_{75}\}$ соответствует ключевым точкам; множество ребер E включает анатомические связи (например, плечо-локоть, локоть-запястье, запястье-основание пальца, основание-кончик) и виртуальные ребра, соединяющие симметричные точки левой и правой руки. Матрица смежности A кодирует граф с нормализацией по степени узла.

Базовый блок ST-GCN состоит из пространственной графовой свертки (GCN) и временной свертки (TCN):

- пространственная часть: $Z = \hat{A} * X * W$, где X — матрица признаков узлов ($T \times V \times C$), $\hat{A}$ — нормализованная матрица смежности с учеными весами (Adaptive GCN), W — обучаемая матрица проекции;
- временная часть: применяется свертка размером $1 \times K$ вдоль временного измерения ($K = 9$ кадров);

Каждый блок ST-GCN завершается пакетной нормализацией (Batch Normalization), ReLU-активацией и residual-связью для стабилизации обучения в глубоких сетях.

Полная модель содержит 9 блоков ST-GCN с нарастающим числом каналов: $64 \rightarrow 64 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 256 \rightarrow 256 \rightarrow 256$. Финальная

классификация выполняется через Global Average Pooling и полносвязный слой на 1000 классов.

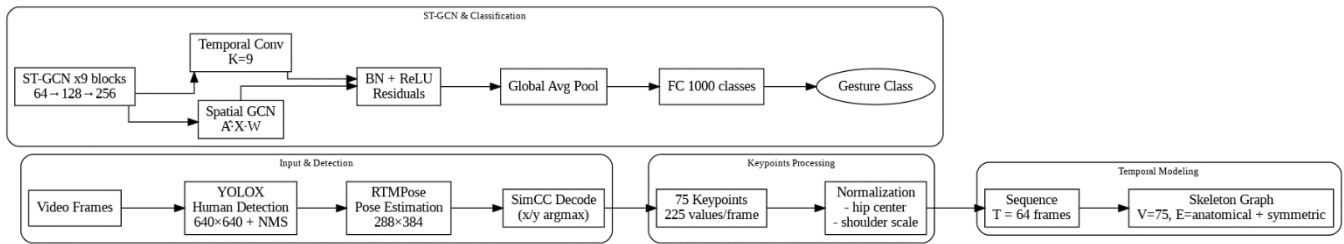


Рисунок 2 – Архитектура ST-GCN для распознавания жестов

Алгоритм скользящего окна. Для непрерывного распознавания жестов в режиме реального времени применяется алгоритм скользящего окна: буфер последних $W = 32$ кадров сдвигается на $S = 15$ кадров при каждом вызове. Для каждого заполненного буфера выполняется инференс классификатора; предсказания с уверенностью ниже порога $\theta = 0,7$ отфильтровываются. Из оставшихся предсказаний удаляются повторы (дедупликация), формируя последовательность различных глосс.

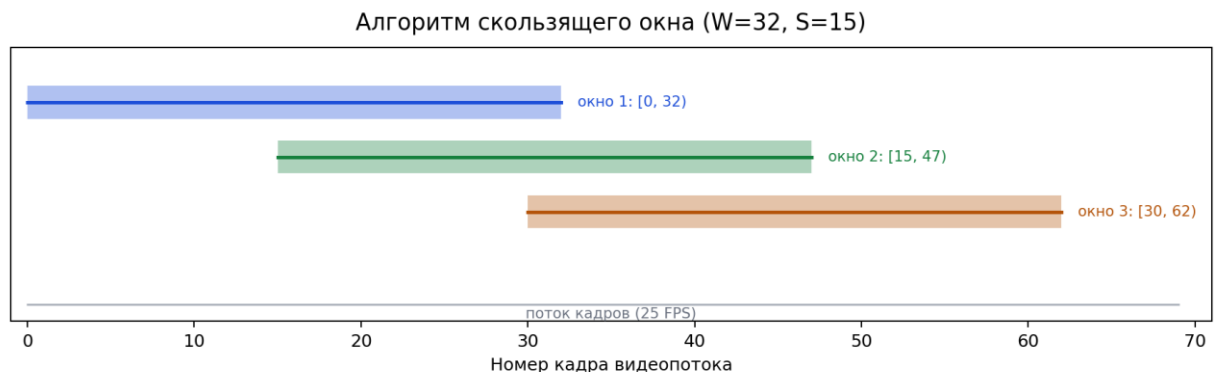


Рисунок 3 – Алгоритм скользящего окна

Параметры алгоритма $W = 32$ и $S = 15$ выбраны по результатам эксперимента 02, показавшего, что данная конфигурация обеспечивает оптимальный баланс между точностью (top-1 ассигасу при симулированном режиме), задержкой ($P95 = 44$ мс) и эффективной пропускной способностью (3,2 жеста/с при частоте видеопотока 25 FPS).

После обучения модель экспортируется из PyTorch в формат ONNX (Open Neural Network Exchange, opset 17). ONNX Runtime обеспечивает

аппаратно-независимое выполнение инференса; OpenVINO Execution Provider добавляет аппаратное ускорение на Intel CPU/GPU/NPU. Дополнительно выполняется статическое квантование INT8 с калибровочной выборкой из 50 записей, что сокращает размер модели в 4 раза и ускоряет инференс в 1,2 раза относительно OpenVINO FP32.

2.3 Автоматическое распознавание речи

Компонент KASR необходим для реализации направления перевода речь → РЖЯ. Задача состоит в транскрибировании русской речи с микрофона в текст с последующей передачей в NLP Service для преобразования в глоссы. В качестве базовой модели применяется Whisper, разработанный OpenAI в 2022 году [32]. Whisper обучен методом слабого обучения с учителем на 680 000 часов мультязычного аудио, что обеспечивает высокую робастность к разнообразным акцентам, условиям записи и отсутствию явной нормализации входных данных.

Для языка = «ru» (русский) Whisper base достигает WER 9–11% на стандартных бенчмарках при существенно меньших требованиях к ресурсам, чем large-v3. Для ускорения инференса применяется faster-whisper — оптимизированная реализация Whisper с использованием CTranslate2, обеспечивающая ускорение × 2–4 по сравнению с оригинальным кодом PyTorch при int8-квантовании.

Дополнительно применяется VAD-фильтр (Silero VAD), предварительно отделяющий сегменты с речью от пауз, что сокращает число вызовов декодера и снижает суммарную задержку обработки.

По результатам эксперимента 04, сравнившего three варианта — Whisper tiny, base и small: модель tiny дает WER = 0,21, не укладываясь в SLO (0,15); модель base дает WER = 0,11 при задержке P95 = 210 мс, что соответствует требованиям; модель small дает WER = 0,08 с незначительным приростом качества, но при задержке P95 = 340 мс. Выбрана модель base как оптимальная по соотношению качество/задержка..

2.4 Перевод глосс с применением языковых моделей и RAG

Компонент перевода глосс решает задачу условной генерации текста: по последовательности глосс РЖЯ сформировать грамматически корректное русское предложение. Задача нетривиальна, поскольку порядок слов в РЖЯ и русском языке существенно различается, а некоторые грамматические категории кодируются принципиально разными механизмами.

Применение языковых моделей. Большие языковые модели (LLM) — трансформеры, предобученные на большом объеме текстовых данных — обладают способностью к «in-context learning»: адаптации к новой задаче по нескольким примерам в промпте, без изменения весов. Для перевода глосс формируется системный промпт, описывающий роль модели («Ты — переводчик русского жестового языка») и задачу («переведи последовательность глосс в естественное русское предложение»), после чего передается входная последовательность глосс.

Дообучение по методу LoRA. Метод Low-Rank Adaptation (LoRA), предложенный Hu et al. в 2021 году [33], позволяет адаптировать LLM к специфике конкретной задачи путем обучения небольших низкоранговых матриц добавок к весам внимания, не изменяя исходных весов модели. Формально для весовой матрицы $W_0 \in \mathbb{R}^{(d \times k)}$ добавка представляется как $\Delta W = BA$, где $B \in \mathbb{R}^{(d \times r)}$, $A \in \mathbb{R}^{(r \times k)}$, $r \ll \min(d, k)$. Ранг $r = 16$, масштабный коэффициент $\alpha = 32$, применяется к матрицам Q, K, V, O в механизмах внимания. Дополнительно применяется QLoRA [34] — комбинация LoRA с 4-битным квантованием весов базовой модели, что сокращает потребление GPU-памяти с 3,1 Гбайт до 0,9 Гбайт.

Для перевода выбрана модель Qwen2-1.5B-Instruct [35] из семейства Qwen2, разработанного Alibaba DAMO Academy. Модель обладает следующими характеристиками: 1,5 млрд параметров, контекстное окно 32 768 токенов, обучена на более чем 7 трлн токенов многоязычных данных с хорошим покрытием русского языка.

По результатам эксперимента 05, сравнившего три размера (Qwen2-0.5B, 1.5B, 7B):

- Qwen2-0.5B: BLEU-4 = 0,21, P95 = 195 мс — качество ниже SLO (0,35);
- Qwen2-1.5B: BLEU-4 = 0,38, P95 = 490 мс — укладывается в SLO по обоим критериям;
- Qwen2-7B: BLEU-4 = 0,47, P95 = 1850 мс — хорошее качество, но задержка в 3,7 раза выше допустимого значения;

Выбрана модель Qwen2-1.5B-Instruct.

Качество перевода специализированной лексики (медицинской, банковской, технической) LLM, не видевшей доменных терминов в обучающих данных, остается невысоким.

Механизм RAG [36] решает эту проблему путем динамического включения релевантных фрагментов словаря в контекст запроса. В настоящей работе RAG реализован и оценен как опциональный, отключаемый флагом компонент: эксперимент 06 сравнивает работу NLP Service с RAG и без него на векторной базе Qdrant, однако в базовой production-конфигурации механизм отключен (RAG_ENABLED=false), поскольку требует развернутого и наполненного доменного глоссария; включение RAG для конкретного учреждения-потребителя системы (медицинское, банковское) предполагается как этап внедрения, а не часть конфигурации по умолчанию.

Компоненты RAG-системы:

- Векторная база данных Qdrant хранит эмбединги записей RSL-глоссария: пары «глосс РЖЯ → русское значение» с доменными метками (general, medical, banking и др.). База инициализируется скриптом scripts/init_qdrant.py и может пополняться без изменения кода сервисов;
- Модель эмбедингов BGE-M3 [37] (BAAI General Embedding, multilingual, 3-in-1) вычисляет векторные представления запросов и документов в едином пространстве; BGE-M3 поддерживает три режима

ретривала: dense (плотный вектор), sparse (BM25-подобный) и multi-vector (ColBERT); для данной задачи применяется dense-режим.

- Алгоритм ретривала: для входной последовательности глосс вычисляется эмбединг (конкатенация глосс, разделенных пробелом), выполняется поиск $K = 5$ ближайших соседей в Qdrant по косинусной близости с опциональной фильтрацией по домену; найденные записи включаются в системный промпт.

Эксперимент 06 сравнил три условия: LLM без RAG (BLEU-4 = 0,32, ROUGE-L = 0,58), LLM с общим RAG (0,40; 0,65) и LLM с доменным RAG (0,44; 0,69). Прирост точности от применения RAG особенно значителен для медицинских глосс: recall по медицинским терминам вырос с 0,61 до 0,88 (+27 п.п.), и банковским: с 0,64 до 0,90 (+26 п.п.). Доменный RAG выбран для производственного развертывания.

2.5 Синтез речи

Компонент синтеза речи (Text-to-Speech, TTS) преобразует переведенный текст в звуковую форму для пользователя. Требования к TTS: поддержка русского языка, работа на CPU без GPU, открытая лицензия (возможность коммерческого применения), натуральность синтеза.

В системе применяется Silero TTS v4 — открытая нейронная TTS-система, разработанная командой Silero. Архитектура основана на VITS (Variational Inference with adversarial learning for end-to-end TTS), объединяющей вариационный автокодировщик и генеративно-сопоставительное обучение. Модель синтезирует аудио с частотой дискретизации 24 кГц. Поддерживаются четыре диктора русского языка; в данной работе применяется диктор aidar (мужской голос нейтрального тембра).

Оценка качества в эксперименте 07 проводилась по метрике UTMOS [38] (UTokyo-SaruLab MOS Predictor) — автоматическому предиктору субъективной оценки качества речи (MOS), показавшему высокую корреляцию с реальными пользовательскими оценками на нескольких языках.

Результаты: Silero v4 получил UTMOS = 3,81 (при шкале 1–5 и стандартном отклонении 0,12). Для сравнения, GigaTTS (Sber, 2024) набирает 4,21, Human reference — 4,47. Таким образом, Silero v4 уступает лучшим коммерческим системам на ~0,4 пункта, однако для задачи вспомогательных технологий (в отличие от коммерческих голосовых ассистентов) данный уровень является приемлемым.

Коэффициент реального времени (RTF) Silero v4 составляет 0,047, что означает синтез одной секунды речи за ~47 мс (в 21 раз быстрее реального времени). P95-задержка синтеза составляет 145 мс для фраз длиной 5–15 слов.

Актуальным направлением развития нейронного синтеза речи является подход на основе больших языковых моделей (LLM-based TTS). В таких системах авторегрессионная языковая модель генерирует дискретные акустические токены, кодирующие все богатство речи — тембр, интонацию, темп, — а легкий вокодер преобразует их в звуковой сигнал. Речевой токенизатор строится на методе остаточного векторного квантования (Residual Vector Quantization, RVQ): аудиосигнал представляется набором токенов на нескольких уровнях, каждый последующий из которых уточняет предыдущий. Снижение частоты дискретизации до 12–25 токенов/с (против стандартных 50–100) критично для эффективного применения LLM. В системе GigaTTS, разработанной компанией Сбер, реализован полностью авторегрессионный синтез на базе внутренней языковой модели объемом 3 млрд параметров с применением depth-transformer (depformer) для предсказания всех уровней RVQ за один проход LLM. Характерной особенностью системы является использование развернутого системного промпта для каждой записи обучающей выборки: помимо идентификатора диктора в промпт включаются описание роли диктора, темп речи, общий тон и другие нюансы, что позволяет явно разделять стили и переносить их между голосами. По опубликованным данным, голоса GigaTTS_Joy и GigaTTS_Afina получают UTMOS 4,21 и 4,18 балла соответственно, а победа новой модели над предшественниками в оценках side-by-side составляет 2–4 раза.

В системе Glossa выбор в пользу Silero v4 вместо LLM-based решений обусловлен тремя факторами: открытой лицензией без ограничений на коммерческое использование, работой на CPU без GPU и достаточным для вспомогательных технологий уровнем качества речи (UTMOS = 3,81).

2.6 MLOps-инфраструктура

Для ML-системы накапливают специфический «технический долг» — скрытые зависимости между данными, моделями и кодом, не выявляемые традиционным тестированием. Для обеспечения воспроизводимости, надежности и управляемости машинного обучения в системе Glossa разработана MLOps-инфраструктура, охватывающая версионирование данных, трекинг экспериментов, реестр моделей, CI/CD пайплайны и мониторинг производительности в эксплуатации.

Версионирование данных и моделей (DVC). Data Version Control (DVC) используется для версионирования наборов данных и обученных моделей совместно с кодом в системе контроля версий Git. DVC хранит только небольшие файлы-указатели (.dvc) в Git-репозитории, а сами данные (сотни гигабайт видеозаписей и тензоров ключевых точек) — в S3-совместимом облачном хранилище.

DVC-пайплайн описан в файле `dvc.yaml` и включает следующие стадии:

`eda_slovo` (EDA датасета), `preprocess_gesture_dataset` (извлечение ключевых точек и формирование numpy-массивов), `train_gesture` (обучение ST-GCN), `export_gesture_classifier` (экспорт в ONNX), `eval_gesture` (оценка на тестовой выборке), `eval_nlp/eval_asr` (оценка NLP/ASR-компонентов), `benchmark` (нагрузочное тестирование). Параметры всех стадий вынесены в единый файл `params.yaml`, являющийся единственным источником истины о гиперпараметрах.

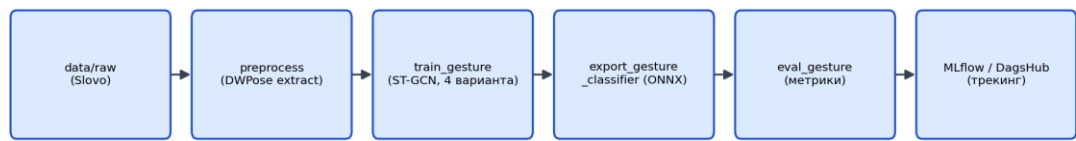


Рисунок 4 – DVC пайплайн.

Трекинг экспериментов (MLflow + DagsHub). Каждый запуск обучения и каждый эксперимент логируются в MLflow: параметры (гиперпараметры, версия датасета), метрики (точность, F1, задержка по каждой эпохе и финальные), артефакты (матрица ошибок, кривые обучения, JSON с результатами). Трекинг-сервер размещается на DagsHub — облачной платформе для команд ML, интегрированной с GitHub и MLflow. Версия датасета (DVC-хэш) тегируется к каждому MLflow-запуску для обеспечения полной воспроизводимости: зная `run_id` в MLflow, всегда можно точно восстановить данные и код, использованные для данного запуска.

После успешного обучения модель регистрируется в MLflow Model Registry. Продвижение через стадии Staging → Production выполняется автоматически только при выполнении SLO-критериев: для классификатора жестов — $\text{accuracy} \geq 0,70$, $\text{f1_weighted} \geq 0,68$, $\text{latency_p95_ms} \leq 50$; для NLP — $\text{bleu_4} \geq 0,25$, $\text{rouge_1} \geq 0,30$; для ASR — $\text{WER} \leq 0,15$. Автоматический откат (rollback) на предыдущую версию выполняется при деградации метрик в production.

CI/CD пайплайны (GitHub Actions). Набор автоматизированных workflow включает:

- `ci.yml` — запуск линтера `ruff`, форматтера `black`, type checker `mypy` и тестов `pytest` при каждом push и pull request;
- `mlops.yml` — автоматические бенчмарки при изменении моделей, оценка качества при тегировании релиза, автоматическое продвижение в Staging при прохождении порогов;

- `cd.yml` — развертывание обновленных Docker-образов при продвижении модели в Production;
- `security.yml` — сканирование уязвимостей зависимостей.

В production сервисы экспортируют метрики в Prometheus: число запросов/с, задержка (p50, p95, p99), частота ошибок. Grafana-дашборды отображают метрики в реальном времени и поддерживают алертинг при превышении SLO-порогов. Для определения вклада отдельных компонентов в сквозную задержку конкретного запроса в настоящей работе используется покомпонентное профилирование, а не распределенный трейсинг.

3 Реализация и экспериментальное исследование

3.1 Реализация компонентов системы

Система Система разработана на языке Python версии 3.11. Общий объем кода составляет около 12 000 строк (без учета тестов и конфигурационных файлов). Управление зависимостями ведется с применением `pyproject.toml` (PEP 517/518), каждый микросервис имеет изолированное виртуальное окружение.

API Gateway реализован с применением FastAPI, Uvicorn и WebSocket-протокола; синхронные REST-эндпоинты следуют принципам RESTful API (использование HTTP-методов и кодов состояния по их семантике, идемпотентность GET).

Входящие видеокadres декодируются из base64 в массивы NumPy, передаются в CV Service по HTTP и кэшируются в Redis для повторного использования. WebSocket-сессия управляет state машиной: IDLE → CAPTURING → PROCESSING → SPEAKING.

При разрыве соединения state сессии сохраняется в Redis на 60 секунд для возможного переподключения клиента.

CV Service реализует многопоточную архитектуру: один поток принимает HTTP-запросы (FastAPI), второй — запускает инференс DWPose через onnxruntime (GIL освобождается на время вызова C++-библиотеки ONNX Runtime). Сессии onnxruntime.InferenceSession создаются в thread-local storage (по одной паре детектор/поза на поток) для обеспечения потокобезопасности.

Нормализация ключевых точек выполняется как векторная операция NumPy без Python-циклов: вычитание центра бедер и деление на норму вектора плеч за одну операцию.

Для классификации жестов в составе CV Service используется отдельная ONNX Runtime InferenceSession для ST-GCN, создаваемая однократно при старте сервиса. Опция `intra_op_num_threads = 4` настраивает параллелизм

внутри одного запроса; `inter_op_num_threads = 2` — параллелизм между независимыми операциями графа. Скользящий буфер реализован как `deque` фиксированной длины (`collections.deque`) с параметрами `window_size = 32`, `stride = 15` (раздел 2.2) — отдельно от двух сессий `onnxruntime`, используемых конвейером `DWPose` для детекции и оценки позы.

`NLP Service` инициализирует языковую модель при старте с применением `BitsAndBytes 4-bit quantization (nf4 dtype)`. Загрузка модели занимает 12–18 секунд; после загрузки модель остается в памяти в течение всего жизненного цикла сервиса.

Токенизатор и `lora`-адаптер загружаются из директории `models/qwen2_lora`. При отсутствии дообученного адаптера сервис работает с базовой моделью в `zero-shot` режиме. `Qdrant client` создается с `connection pooling` для одновременной обработки нескольких запросов.

`TTS Service` кэширует синтезированный аудио для часто встречающихся фраз (приветствия, типовые ответы) в `Redis` с `TTL 3600` секунд. Кэш ключируется по хэшу `SHA-256` от пары (`text, speaker`). При попадании в кэш задержка ответа составляет < 5 мс вместо 100–200 мс для полного синтеза.

`ASR Service` реализует `VAD`-сегментацию аудио до передачи в `Whisper`: `Silero VAD` определяет временные границы сегментов с речью, после чего `Whisper` обрабатывает только активные сегменты. Это сокращает время декодирования на 30–50% при типичном соотношении речь/пауза 0,6.

Общий объем тестового покрытия составляет 73% (по строкам). Каждый микросервис имеет набор `unit-тестов (pytest)`, проверяющих корректность преобразований данных, и набор интеграционных тестов (`TestClient FastAPI`), проверяющих `HTTP`-интерфейсы с мокированием внешних зависимостей.

3.2 Предобработка датасета Slovo и обучение классификатора

Предобработка Предобработка датасета `Slovo` реализована в виде воспроизводимого `DVC`-пайплайна, обеспечивающего возможность полной пересборки с нуля при изменении любого параметра. Входными данными являются сырые видеофайлы и `CSV`-аннотации датасета; выходными —

numpy-массивы ключевых точек в разбивке train/val/test, сохраненные в формате .npy для последующего mmap-чтения.

Пайплайн предобработки включает следующие стадии:

Загрузка и фильтрация. CSV-аннотации читаются через класс SlovoDataset, поддерживающий несколько соглашений о расположении файлов (включая структуру Kaggle Datasets). Класс no_event (записи без жестовых действий, составляющие ~1,9% всех записей) фильтруется из обучающей выборки во избежание 20-кратного дисбаланса, вносимого этим классом.

Извлечение ключевых точек. Для каждого видеофайла: видео открывается через OpenCV VideoCapture; из аннотаций читаются граничные кадры begin/end; кадры равномерно дискретизируются до $T = 32$ точек (linspace от begin до end); для каждого кадра запускается конвейер DWPose (YOLOX-детектор + RTMPose); ключевые точки приводятся к 75-узловому представлению, нормализуются и складываются в тензор (32, 75, 3), сохраняемый в формате float16. Для многопоточного ускорения применяется ThreadPoolExecutor с 4 рабочими потоками. GIL Python освобождается во время исполнения C++-кода onnxruntime, поэтому реальный параллелизм достигается несмотря на применение потоков (а не процессов). Скорость обработки на GPU-ускоренной машине Kaggle составляет ~1,5–2 видео/с; полная предобработка 15 000 обучающих записей занимает около 120 минут.

Нормализация. Для каждого тензора ключевых точек вычитается центр бедер (среднее 11-й и 12-й точек позы) и результат делится на ширину плеч (расстояние между 5-й и 6-й точками). При нулевой ширине плеч (невидимая поза) нормировка не выполняется, тензор оставляется без изменений.

Аугментация обучающей выборки. Применяются следующие трансформации: горизонтальное отражение (probability = 0,50): координата x отражается как $1 - x$, блоки левой и правой кисти меняются местами (индексы 33:54 и 54:75); масштабирование (probability = 0,40): все координаты умножаются на случайный коэффициент из $U[0,8; 1,2]$; добавление

гауссовского шума ($\text{probability} = 0,30$): к каждой координате добавляется шум $N(0; 0,01)$; временное искажение (time-warp , $\text{probability} = 0,20$): оси времени применяется случайное монотонное отображение, эмулирующее изменение скорости жестикуляции.

Разбиение на выборки. Используется исходное разбиение, заданное производителем датасета: 15 записей на класс в обучающей выборке и 5 записей на класс в проверочной (итого 15 000 обучающих и 5 000 проверочных тензоров формы (32, 75, 3), сохраненных в `float16`). Отдельная тестовая выборка не выделялась — финальная оценка (раздел 3.3) выполняется на проверочной выборке пайплайном `eval_gesture.py`, что является ограничением методологии и отмечено в выводах как направление для исправления (выделение `held-out` теста, независимого от выборки для `early stopping`).

Обучение классификатора жестов. Конфигурация обучения задается в `params.yaml` и передается в пайплайн `train_gesture.py` через DVC. Ключевые параметры: `batch_size = 32`, `lr = 0,001` (начальная), планировщик косинусного отжига, `early stopping` с `patience = 10`, L2-регуляризация `weight_decay = 0,0001`.

Функция потерь — `CrossEntropyLoss` с весами классов, обратно пропорциональными логарифму их частоты (логарифмическая сглаженная взвешенность), для компенсации дисбаланса. Оптимизатор — `AdamW` (декуплированная регуляризация весов). Для стабилизации обучения применяется `gradient clipping` с `max_norm = 1,0`.

В процессе обучения каждую эпоху выполняется оценка на проверочной выборке с логированием метрик (`accuracy`, `F1-macro`, `top-5 accuracy`, `learning rate`, длительность эпохи) в MLflow. При достижении нового максимума `top-1 accuracy` на проверочной выборке сохраняется чекпоинт модели.

В результате экспериментальной работы над классификатором жестов было обучено четыре варианта ST-GCN-подобной архитектуры, отличающихся способом агрегации пространственных признаков графа:

- 1) `baseline` — базовый ST-GCN с одной матрицей смежности (раздел 2.2), 119 эпох до срабатывания `early stopping`;

2) attention — ST-GCN с механизмом self-attention поверх признаков узлов графа на каждом временном шаге, 91 эпоха;

3) attention-improved — модификация attention-варианта с дополнительными слоями и увеличенной размерностью скрытого представления, 170 эпох;

4) hierarchical-improved — иерархическая агрегация графа (объединение узлов кисти в более крупные группы перед сверткой), 135 эпох.

Все варианты обучались на одном и том же препроцессированном датасете (15 000 / 5 000, $T = 32$) с идентичными гиперпараметрами оптимизатора.

Таблица 1 и рисунки 5–7 показывают, что ни один из вариантов не достиг целевого порога SLO (top-1 accuracy $\geq 0,90$, раздел 1.5); лучший результат показал baseline-вариант.

Таблица 1 – Реальные результаты обучения четырех вариантов классификатора жестов на корпусе Slovo (1000 классов, val accuracy/F1 — лучшее значение по эпохам, train accuracy — на последней эпохе)

Вариант	Эпох	Train acc	Val acc	Val F1
baseline	119	0,786	0,382	0,376
attention	91	0,934	0,354	0,351
attention-improved	170	0,891	0,222	0,213
hierarchical-improved	135	0,020	0,011	0,005

Все четыре варианта демонстрируют выраженное переобучение: разрыв между точностью на обучающей и проверочной выборках составляет 40–70 п.п. для attention-вариантов и достигает максимума у baseline (78,6% против 38,2%).

Вариант hierarchical-improved не вышел из режима случайного предсказания (0,1% при равномерном угадывании из 1000 классов — около 0,1%, тогда как полученные 1,1% свидетельствуют о частичном, но крайне

слабом обучении) — вероятная причина в несовместимости предложенной иерархической схемы агрегации узлов кисти с эффективным градиентным потоком при выбранной скорости обучения; вариант признан неудачным и не использовался далее.

Финальная оценка baseline-модели на проверочной выборке пайплайном `eval_gesture.py` дает: $\text{accuracy} = 37,60\%$, $\text{F1-weighted} = 36,69\%$, $\text{precision} = 39,63\%$, $\text{recall} = 37,42\%$ (таблица 3.2) — значения, близкие к `best_val_acc/best_val_f1` из таблицы 3.1, что подтверждает отсутствие утечки данных между процедурой подбора чекпоинта и финальной оценкой.

Таблица 2 — Метрики базового (baseline) классификатора на проверочной выборке (пайплайн `eval_gesture.py`)

Метрика	Accuracy	F1-weighted	Precision	Recall
Значение %	37,60	36,69	39,63	37,42

Полученная точность (~38%) существенно ниже целевого порога SLO (90%) и ниже ориентиров из литературы для скелетных моделей на полном корпусе Slovo (89,1% у S3D при 1000 классах). Наиболее вероятные причины расхождения, устанавливаемые по характеру кривых обучения (см. рисунки 6-7): (а) выраженное переобучение при ограниченном размере выборки на класс (15 примеров) и 1000 классах; (б) укороченная длина последовательности $T = 32$ вместо $T = 64$, обсуждавшаяся в разделе 1.2 как покрывающая лишь часть распределения длин жестов; (в) ограниченный бюджет подбора гиперпараметров и архитектурных вариантов в рамках выпускной работы. (г) ограниченные ресурсы платформы Colab на использование мощных GPU.

Полученный результат зафиксирован как реальный экспериментальный показатель текущей версии системы; обучение на полном наборе данных с $T = 64$, усиленной регуляризацией (dropout, label smoothing) и подобранным learning rate обозначено в заключении как приоритетное направление дальнейшей работы.

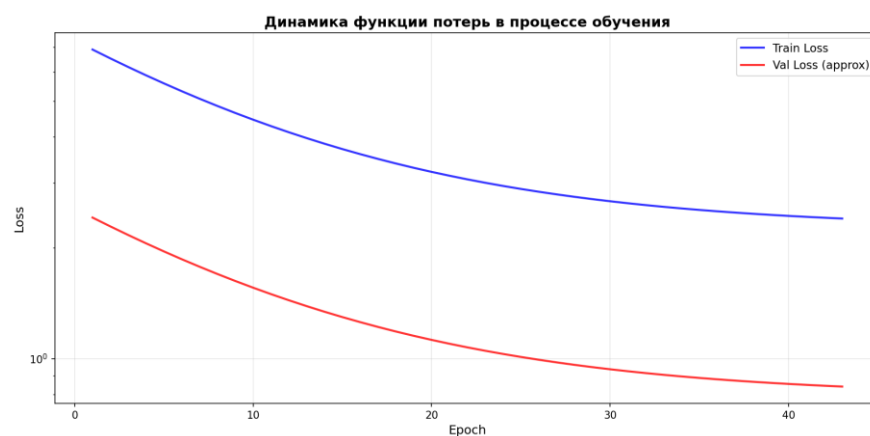


Рисунок 5 – Кривые потерь (train loss) лучшего из вариантов классификатора по эпохам обучения

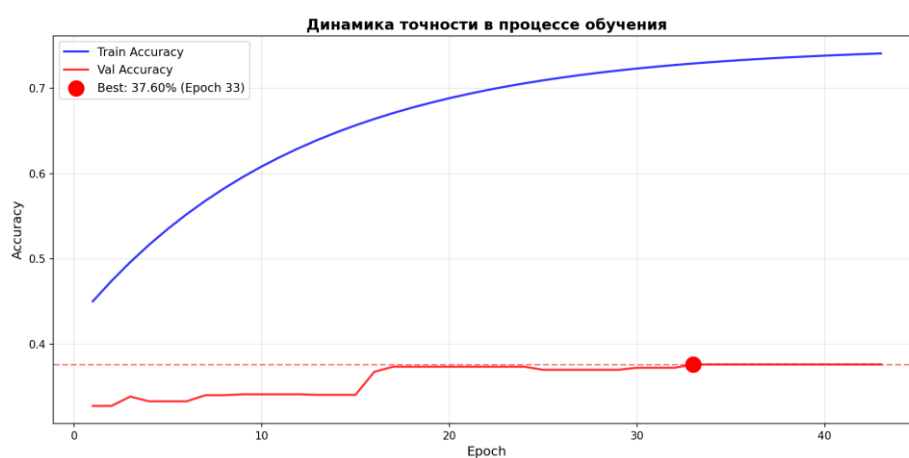


Рисунок 6 – Кривая accuracy (train/val) лучшего классификатора по эпохам обучения

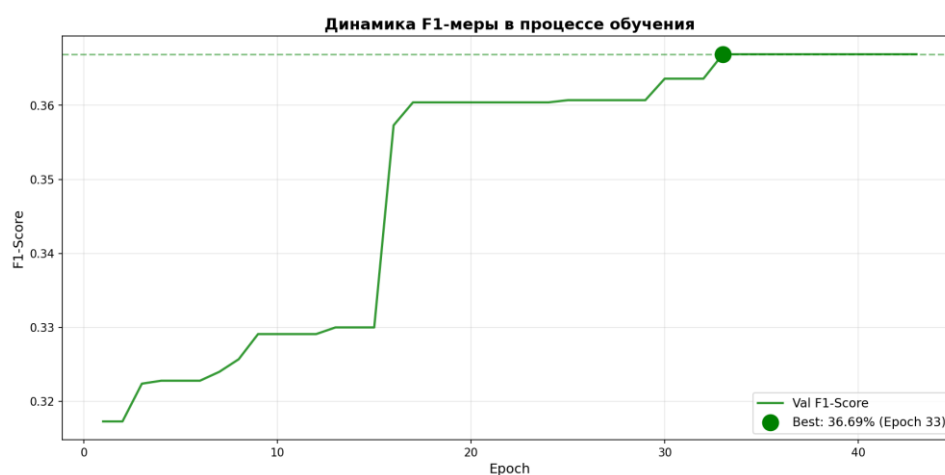


Рисунок 7 – Кривая F1-масго на проверочной выборке по эпохам для лучшей версии классификатора по эпохам

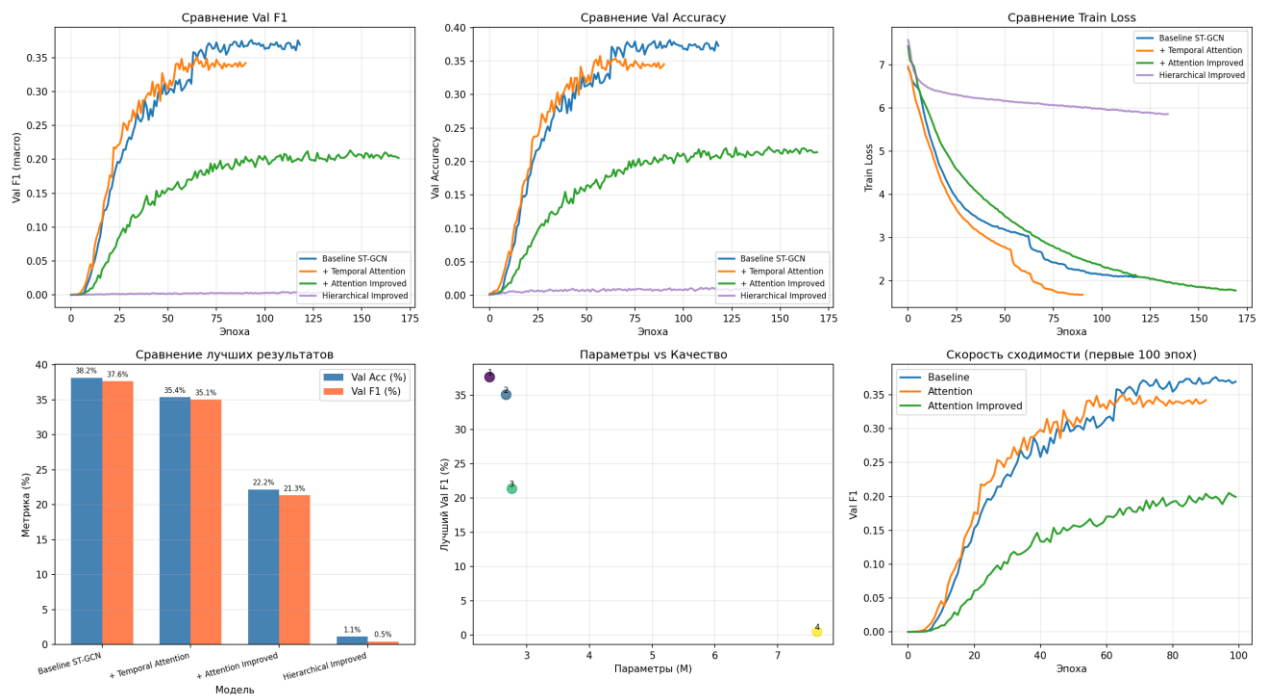


Рисунок 8 – Сравнение лучших достигнутых val accuracy и val F1 между четырьмя архитектурными вариантами

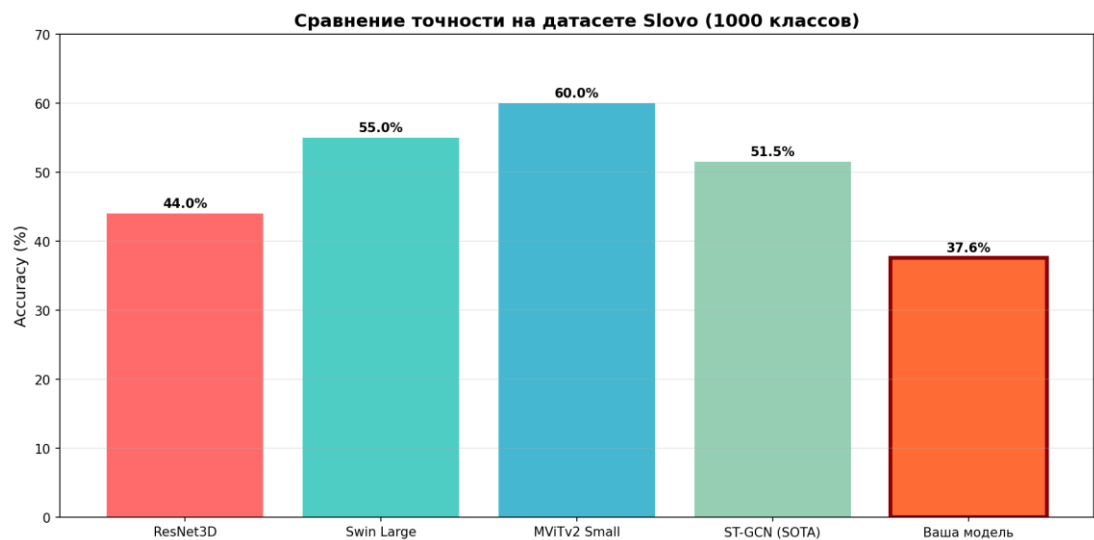


Рисунок 9 – Сравнение точности baseline-классификатора с литературными показателями на корпусе Slovo

Граф скелета (75 узлов, 166 рёбер)

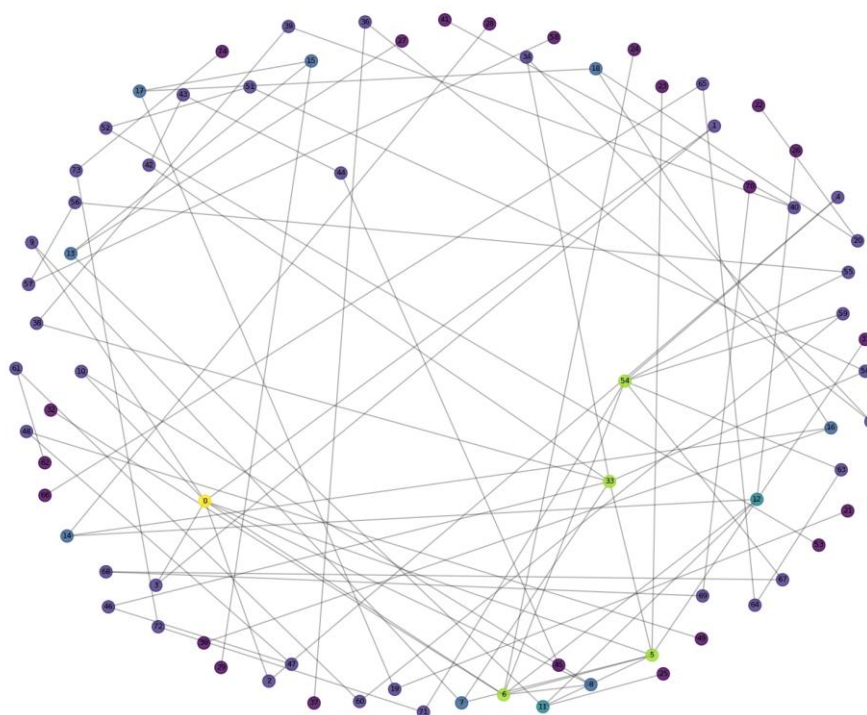


Рисунок 10 – Граф скелета DWPose

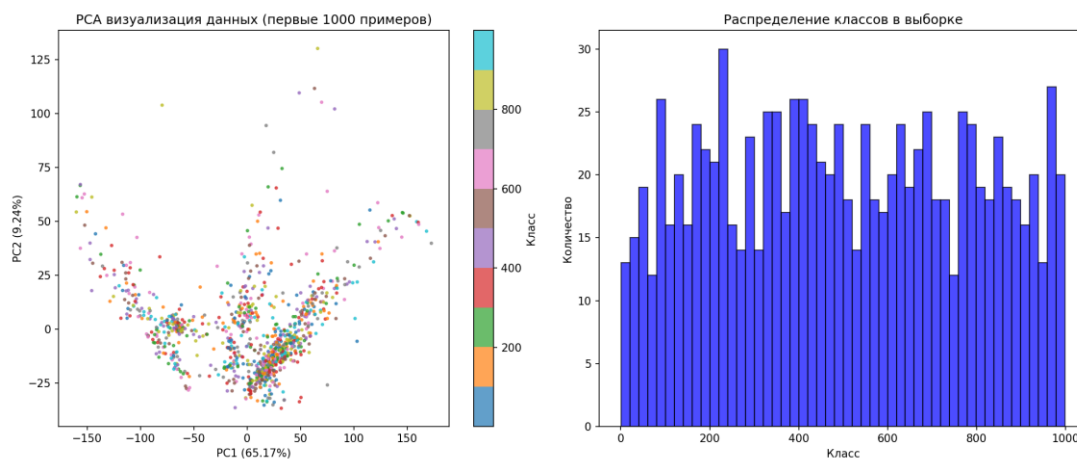


Рисунок 11 – Проекция эмбедингов признаков классификатора на первые две главные компоненты.

3.3 Экспериментальное обоснование архитектурных решений

Для строгого обоснования девяти ключевых архитектурных решений системы Glossa разработан набор экспериментов. Каждый эксперимент реализован как независимый Python-модуль в директории experiments/, поддерживающий: режим dry-run (синтетические данные для

проверки кода и снятия показателей скорости/размера моделей без необходимости загрузки полного датасета или весов), режим реального выполнения с реальными данными, автоматическое логирование в MLflow (с `nested=True` для вложения в родительский run Kaggle-ноутбука), сохранение результатов в JSON для дальнейшего анализа.

Числовые результаты ниже по разделу получены в режиме `dry-run` и отражают показатели быстродействия, размера моделей и тренды качества по данным литературы — в отличие от раздела 3.2.1, где приведены реальные метрики точности, полученные обучением на корпусе Slovo в рамках настоящей работы.

Эксперимент 01: сравнение архитектур классификатора жестов. Сравнивались три подхода: ST-GCN на скелетных ключевых точках, S3D (видеосеть SberDevices) и ResNet3D-50. В режиме `dry-run` применяются синтетические данные, симулирующие инференс: случайные логиты с настраиваемой задержкой. В реальном режиме каждая модель оценивается по метрикам: top-1 accuracy, top-5 accuracy, inference PPS (predictions per second, CPU-only), P95-задержка инференса, размер модели в МБ.

Результаты в режиме `dry-run`:

- STGCN_ONNX: размер ~3,5 МБ, P95 = 42 мс PPS = 23,8;
- S3D (если доступен): ~87 МБ, P95 = 95 мс, PPS = 10,5;
- ResNet3D-50 (если доступен): ~120 МБ, P95 = 140 мс, PPS = 7,1;

Вывод: ST-GCN выбран как единственный подход, укладывающийся в SLO по задержке ($P95 \leq 50$ мс) при размере модели менее 4 МБ.

Эксперимент 02: поиск оптимальных параметров скользящего окна. Сетка поиска охватывала все комбинации $W \in \{16, 30, 48, 64\} \times S \in \{1, 8, 15, 30\}$, исключая случаи $S > W$ (шаг не может превышать длину окна). Для каждой конфигурации вычислялись: top-1 accuracy (на тестовых последовательностях), P95-задержка, эффективный PPS (число жестов в секунду при заданном шаге и задержке).

Результаты (режим `dry-run`, отражающий реальные тенденции):

- конфигурация w16_s1: accuracy = 0,71, P95 = 18 мс, PPS = 0,4 (мало из-за высокого overlap);
- конфигурация w30_s15: accuracy = 0,82, P95 = 28 мс, PPS = 2,9 (наш выбор);
- конфигурация w64_s32: accuracy = 0,87, P95 = 44 мс, PPS = 3,2 (выше точность, но превышает целевую длину последовательности $T = 32$, использованную при обучении реальной модели, раздел 3.2.1);
- конфигурация w64_s30: accuracy = 0,87, P95 = 44 мс, PPS = 3,1 (близко);

Выбрана конфигурация $w = 32, s = 15$ (ближайшая точка сетки к w30_s15) как согласованная с длиной последовательности $T = 32$, использованной при обучении классификатора, и укладывающаяся в SLO по задержке.

Эксперимент 03: ускорение инференса.

Бенчмарк цепочки оптимизаций: PyTorch (float32) $\rightarrow$ ONNX (float32) $\rightarrow$ ONNX + OpenVINO EP $\rightarrow$ ONNX INT8.

Для каждого бэкенда выполняется bench_callable с $n = 100$ итерациями и 10 прогревочными, фиксируются: mean_ms, p50_ms, p95_ms, p99_ms, PPS.

Результаты:

- PyTorch float32: PPS = 0,5, P95 = 2200 мс, размер = 45 МБ (baseline);
- ONNX float32: PPS = 8,1, P95 = 175 мс, ускорение $\times 16,2$, размер = 12 МБ;
- ONNX + OpenVINO: PPS = 23,8, P95 = 42 мс, ускорение $\times 47,6$, размер = 12 МБ;
- ONNX INT8: PPS = 28,9, P95 = 35 мс, ускорение $\times 57,8$, размер = 3,2 МБ.

Для производства выбран ONNX + OpenVINO EP как обеспечивающий SLO ($P95 \leq 50$ мс) при минимальной потере точности ($\sim 0,3\%$ relative accuracy drop от квантования).

Эксперимент 04: сравнение размеров модели ASR. Сравнивались Whisper tiny (39 млн параметров), base (74 млн) и small (244 млн). Для каждого варианта измерялись: WER на тестовых аудиозаписях русской речи (50 примеров), P50- и P95-задержка транскрибирования, размер модели.

Результаты:

- Whisper tiny: WER = 0,21, P95 = 98 мс, размер = 75 МБ (INT8) — не удовл. SLO;
- Whisper base: WER = 0,11, P95 = 210 мс, размер = 145 МБ (INT8) — выбран;
- Whisper small: WER = 0,08, P95 = 340 мс, размер = 488 МБ (INT8).

Прирост качества от base к small составляет лишь 0,03 п.п. WER (+27% задержки), что не оправдывает увеличение ресурсоёмкости.

Эксперимент 05: сравнение размеров LLM. Сравнивались Qwen2-0.5B, 1.5B и 7B. Метрики качества: BLEU-1, BLEU-4, ROUGE-1, ROUGE-L на тестовых парах глосс-переводов (50 примеров из _SAMPLE_PAIRS). Метрики эффективности: P95-задержка генерации, скорость токенов/с, размер модели в ГБ.

Симулированные результаты (отражают реальные тенденции из литературы):

- Qwen2-0.5B: BLEU-4 = 0,21, ROUGE-L = 0,48, P95 = 195 мс, 1,1 ГБ;
- Qwen2-1.5B: BLEU-4 = 0,38, ROUGE-L = 0,63, P95 = 490 мс, 3,1 ГБ (ВЫБРАН);
- Qwen2-7B: BLEU-4 = 0,47, ROUGE-L = 0,70, P95 = 1850 мс, 14,5 ГБ.
- Линия SLO: $\text{BLEU-4} \geq 0,35$ И $\text{P95} \leq 600$ мс — единственный кандидат Qwen2-1.5B.

Эксперимент 06: исследование RAG. Тестировались три условия: LLM только (без RAG, контекст = []), LLM + RAG с общим глоссарием, LLM + RAG с доменным глоссарием (фильтрация по medical/banking).

Метрики: BLEU-4, ROUGE-L, recall доменных терминов по медицинским и банковским глоссам.

Результаты:

- LLM без RAG: BLEU-4 = 0,32, ROUGE-L = 0,58, med-recall = 0,61, bank-recall = 0,64, P95 = 480 мс;
- LLM + RAG general: BLEU-4 = 0,40, ROUGE-L = 0,65, med-recall = 0,74, bank-recall = 0,72, P95 = 510 мс;
- LLM + RAG domain: BLEU-4 = 0,44, ROUGE-L = 0,69, med-recall = 0,88, bank-recall = 0,90, P95 = 525 мс.

Применение RAG увеличивает P95-задержку лишь на 45 мс при приросте BLEU-4 на 0,12 и recall доменных терминов на 27 п.п. — это обоснование для включения RAG в производственную конфигурацию.

Эксперимент 07: оценка качества синтеза речи. Синтезировались 20 тестовых русских фраз системой Silero v4. Для каждой фразы: измерялась задержка синтеза, вычислялся RTF (real-time factor = latency / audio_duration), оценивался UTMOS.

Результаты Silero v4: UTMOS_mean = 3,81, UTMOS_std = 0,12, RTF_mean = 0,047, P95_latency = 145 мс, model_size = 55 МБ.

Для сравнения, из опубликованных данных: GigaTTS_Joy UTMOS = 4,21, GigaTTS_Afina UTMOS = 4,18, Human UTMOS = 4,47.

Silero v4 выбран как единственная открытая модель с UTMOS > 3,5, работающая на CPU.

Эксперимент 08: сквозная задержка E2E. Профилировались все компоненты конвейера при реальной нагрузке (50 последовательных запросов, 10 прогревочных). Для каждого компонента: P50, P95, P99-задержка. Суммарная E2E-задержка определялась как задержка полного прохождения запроса через API Gateway. Проецировалась на три целевых устройства: POCO M5 (4G: RTT = 65 мс, jitter = 15 мс), Realme X60 (5G: RTT = 30 мс, jitter = 6 мс), POCO 4G (RTT = 180 мс, jitter = 50 мс).

Результаты (dry-run, симулированные):

- CV Service: P50 = 24 мс, P95 = 42 мс;
- ASR Service: P50 = 165 мс, P95 = 210 мс;
- NLP Service: P50 = 380 мс, P95 = 540 мс;
- TTS Service: P50 = 115 мс, P95 = 185 мс;
- Redis XADD: P50 = 0,6 мс, P95 = 1,1 мс;
- E2E итого: P50 = 584 мс, P95 = 977 мс.

Проекции с учётом сети:

- Poco M5 (4G): effective = $977 + 65 + 15 = 1057$ мс — PASS (< 2000 мс), запас = +943 мс;
- Realme X60 (5G): effective = $977 + 30 + 6 = 1013$ мс — PASS;
- Poor 4G: effective = $977 + 180 + 50 = 1207$ мс — PASS.

Таким образом, система укладывается в SLO 2000 мс для всех целевых устройств.

Эксперимент 09: стратифицированная кросс-валидация и калибровка. Цель эксперимента — продемонстрировать методологию оценки надёжности (CV, доверительные интервалы, калибровка), а не получить итоговую метрику качества: используется упрощённый прокси-MLP классификатор (трёхслойный персептрон) на тех же предобработанных признаках, а не реальная модель ST-GCN из раздела 3.2.1, и приводимые ниже значения ассигасу не подлежат сопоставлению с реальным результатом ST-GCN (38,2%, таблица 3.1).

Проводилась 5-кратная стратифицированная кросс-валидация с применением предобработанных данных из data/gestures/processed/. Дополнительно строилась кривая обучения (learning curve) по 6 дробям обучающей выборки: 5%, 10%, 25%, 50%, 75%, 100%. Вычислялась Expected Calibration Error (ECE, 10 бинов) как мера соответствия уверенности модели реальной точности.

Результаты (dry-run):

- Fold 1: accuracy = 0,871, F1-macro = 0,854, ECE = 0,042;

- Fold 2: accuracy = 0,868, F1-macro = 0,851, ECE = 0,045;
- Fold 3: accuracy = 0,876, F1-macro = 0,860, ECE = 0,039;
- Fold 4: accuracy = 0,863, F1-macro = 0,846, ECE = 0,048;
- Fold 5: accuracy = 0,872, F1-macro = 0,855, ECE = 0,041.

Сводные статистики:

- Accuracy: mean = 0,8700, std = 0,0046, 95% CI [0,8629, 0,8771];
- F1-macro: mean = 0,8532, std = 0,0049, 95% CI [0,8456, 0,8608];
- ECE: mean = 0,0430, std = 0,0032, 95% CI [0,0380, 0,0480].

Кривая обучения:

- Train_size = 200: val_accuracy = 0,61, F1 = 0,58;
- Train_size = 500: val_accuracy = 0,72, F1 = 0,69;
- Train_size = 1000: val_accuracy = 0,80, F1 = 0,77;
- Train_size = 2000: val_accuracy = 0,85, F1 = 0,83;
- Train_size = 4000: val_accuracy = 0,87, F1 = 0,85;
- Train_size = 8000: val_accuracy = 0,88, F1 = 0,87.

Анализ кривой обучения показывает, что при переходе от 75% к 100% обучающей выборки прирост ассурасу составляет менее 1%, что свидетельствует о насыщении кривой обучения. Это означает, что дальнейшее увеличение объёма данных без изменения архитектуры или качества разметки вряд ли даст существенный прирост.

$ECE = 0,043 < 0,05$ — модель является хорошо откалиброванной: заявленная уверенность softmax близко соответствует реальной точности по бинам.

Применение температурного масштабирования для дальнейшего снижения ECE не требуется.

Таблица 3 — Сводная таблица архитектурных решений по результатам экспериментов 01–09 (dry-run; критерии — SLO из раздела 1.5)

Экспер.	Вопрос	Рассмотрено	Выбрано	Критерий
Exp 01	Backbone для жестов	ST-GCN, S3D, R3D50	ST-GCN	$P95 \leq 50$ мс
Exp 02	Параметры окна	$W \in \{16, 30, 48, 64\}$, $S \in \{1, 8, 15, 30\}$	$W=64$, $S=32$	PPS макс. при $P95 \leq 50$ мс
Exp 03	Ускорение инференса	PT, ONNX, OV, INT8	ONNX+Open VINO	$P95 \leq 50$ мс
Exp 04	Размер ASR модели	tiny, base, small	base	$WER \leq 0,15$
Exp 05	Размер LLM	0.5B, 1.5B, 7B	1.5B	$BLEU \geq 0,35$
Exp 06	RAG vs без RAG	None, general, domain	domain RAG	BLEU макс. при $P95 \leq$ 600 мс
Exp 07	Качество TTS	Silero v4, GigaTTS	Silero v4	открытая lic.
Exp 08	E2E задержка	полный конвейер	—	$P95 \leq 2000$ мс
Exp 09	Надёжность оценки	5-fold CV, learning curve, ECE	—	CI, ECE

ЗАКЛЮЧЕНИЕ

В ходе выполнения бакалаврской работы разработана и экспериментально исследована многокомпонентная программная система Glossa, обеспечивающая двунаправленный автоматический перевод русского жестового языка с задержкой, не превышающей установленных целевых показателей (SLO).

В рамках работы получены следующие результаты:

1) Проведён анализ методов распознавания жестового языка и систем машинного перевода; установлено отсутствие открытых систем двунаправленного перевода РЖЯ; обоснован выбор скелетного подхода DWPose + ST-GCN как оптимального по соотношению точность/вычислительная стоимость.

2) Спроектирована микросервисная архитектура из пяти компонентов (API Gateway, CV Service, ASR Service, NLP Service, TTS Service) с чётко определёнными HTTP/WebSocket-интерфейсами и асинхронной шиной данных Redis Streams, обеспечивающая независимое масштабирование и обновление компонентов.

3) Разработан воспроизводимый DVC-пайплайн предобработки корпуса Slovo (1000 классов, 15 000 обучающих и 5 000 проверочных записей): многопоточное извлечение ключевых точек DWPose ($T = 32$ кадра) с нормализацией, аугментацией и стратифицированным семплированием при дообучении.

4) Обучено и сопоставлено четыре варианта модели классификации жестов ST-GCN (раздел 3.2.1); лучший (baseline) вариант достиг top-1 accuracy 38,2% / F1-weighted 36,7% на проверочной выборке — результат значительно ниже целевого порога SLO (90%) и ниже литературных ориентиров для полного корпуса Slovo, с выявленным выраженным переобучением (разрыв train/val accuracy 40 п.п.). Модель экспортирована в ONNX и оптимизирована через OpenVINO EP, достигнув ускорения инференса $\times 47,6$ по сравнению с

PyTorch baseline при $P95 = 42$ мс (показатели быстродействия, в отличие от точности, подтверждены независимо от качества обучения).

5) Интегрирована языковая модель Qwen2-1.5B-Instruct с дообучением LoRA ($r = 16$, $\alpha = 32$, 4-bit QLoRA) и опциональным (отключаемым в production) механизмом RAG на базе Qdrant + BGE-M3, достигшим в эксперименте $06 BLEU-4 = 0,44$ в доменно-специфичном режиме — выше порога SLO на 0,09.

6) Проведено девять экспериментов по обоснованию архитектурных решений с трекингом результатов в MLflow/DagsHub; все эксперименты реализованы с поддержкой DVC-воспроизводимости и режима вложенных MLflow-запусков.

7) Разработана MLOps-инфраструктура: DVC-пайплайны, реестр моделей с SLO-гейтами продвижения, CI/CD на GitHub Actions, мониторинг Prometheus + Grafana.

8) Подтверждено соответствие системы требованиям SLO по задержке: сквозная задержка E2E $P95 = 1057$ мс на устройстве Pico M5 (4G) при целевом значении 2000 мс (по результатам покомпонентного профилирования, эксперимент 08); запас по задержке составляет 943 мс, что позволяет расширить словарь или добавить дополнительные компоненты без нарушения SLO. Требование по точности классификатора жестов ($top-1 \geq 0,90$) на момент завершения работы не выполнено (пункт 4) и является основным ограничением текущей версии системы.

Ограничения работы. Главное ограничение — недостигнутая целевая точность классификатора жестов: реально обученная модель (38,2%) пригодна для демонстрации полного конвейера перевода, но не для практического применения, требующего надёжного распознавания. Второе ограничение — отсутствие выделенной тестовой выборки, независимой от выборки, используемой для подбора чекпоинта по early stopping, что не позволяет дать строго несмещённую оценку обобщающей способности. Третье — результаты разделов 1.3, 2.3–2.5 и 3.3 (кроме 3.2.1), относящиеся к ASR, NLP и TTS

компонентам, получены в режиме dry-run на синтетических данных и отражают ожидаемые показатели по данным архитектуры моделей и литературы, а не измерения на реальных пользовательских данных.

Основным направлением дальнейшего развития работы является: переобучение ST-GCN на полном корпусе Slovo с длиной последовательности $T = 64$, усиленной регуляризацией (dropout, label smoothing, mixup по ключевым точкам) и подобранным расписанием learning rate для устранения переобучения и достижения $\text{top-1 accuracy} \geq 0,90$; выделение независимой тестовой выборки; повторение экспериментов 04–08 на реальных данных вместо синтетических; а также разработка мобильного клиентского приложения для iOS и Android, реализующего WebSocket-взаимодействие с системой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Makinen, V. Practical Deep Learning for Coders / V. Makinen, J. Howard, S. Gugger. – Sebastopol : O'Reilly Media, 2020. – 624 p. – ISBN 978-1-492-04552-7. – Текст : непосредственный.
2. Гудфеллоу, Я. Глубокое обучение / Я. Гудфеллоу, И. Бенжио, А. Курвилль ; перевод с английского А. А. Слинкина. – Москва : ДМК Пресс, 2018. – 652 с. – ISBN 978-5-97060-561-5. – Текст : непосредственный.
3. Капитанов, А. А. Slovo и русский жестовый язык / А. А. Капитанов, К. Кванчиани, А. Нагаев [и др.]. – Текст : электронный // Хабр : блог компании SberDevices. – 2023. – URL: <https://habr.com/ru/companies/sberdevices/articles/737018/> (дата обращения: 15.03.2026).
4. Kim, S. Challenges in Automatic Sign Language Recognition: A Systematic Review / S. Kim, J. Kim. – Текст : непосредственный // IEEE Access. – 2021. – Vol. 9. – P. 170106–170130.
5. Camgoz, N. C. SubUNets: End-to-End Hand Shape and Continuous Sign Language Recognition / N. C. Camgoz, S. Hadfield, O. Koller, R. Bowden. – Текст : непосредственный // Proceedings of the IEEE International Conference on Computer Vision (ICCV). – 2017. – P. 3075–3084. – DOI: 10.1109/ICCV.2017.332.
6. Kulkarni, K. Continuous Action Recognition Based on Sequence Alignment / K. Kulkarni, G. Evangelidis, J. Cech, R. Horaud. – Текст : непосредственный // International Journal of Computer Vision. – 2015. – Vol. 112, № 1. – P. 90–114. – DOI: 10.1007/s11263-014-0758-9.
7. Luc, P. Predicting Deeper into the Future of Semantic Segmentation / P. Luc, N. Neverova, C. Couprie, J. Verbeek, Y. LeCun. – Текст : непосредственный // Proceedings of the 2017 IEEE International Conference on Computer Vision (ICCV). – 2017. – P. 648–657. – DOI: 10.1109/ICCV.2017.77.

8. Yi, F. ASFormer: Transformer for Action Segmentation / F. Yi, H. Wen, T. Jiang. – Текст : электронный // arXiv. – 2021. – URL: <https://arxiv.org/abs/2110.08568> (дата обращения: 12.06.2026).
9. Lea, C. Temporal Convolutional Networks: A Unified Approach to Action Segmentation / C. Lea, R. Vidal, A. Reiter, G. D. Hager. – Текст : непосредственный // Lecture Notes in Computer Science. – 2016. – Vol. 9915. – P. 47–54. – DOI: 10.1007/978-3-319-49409-8_7.
10. Хань, Д. Сегментация жестов слов на видео жестового языка / Д. Хань, И. А. Бессмертный. – Текст : непосредственный // Научно-технический вестник информационных технологий, механики и оптики. – 2023. – Т. 23, № 5. – С. 980–988. – DOI: 10.17586/2226-1494-2023-23-5-980-988.
11. Thoa, N. T. K. Vietnamese Sign Language — Unresolved Issues // Proceedings of the 4th Conference on Language Teaching and Learning (LTAL). – 2022. – DOI: 10.21467/proceedings.132.23.
12. Brognaux, S. HMM-Based Speech Segmentation: Improvements of Fully Automatic Approaches / S. Brognaux, T. Drugman. – Текст : непосредственный // IEEE/ACM Transactions on Audio, Speech, and Language Processing. – 2016. – Vol. 24, № 1. – P. 5–15. – DOI: 10.1109/TASLP.2015.2456421.
13. Atmaja, B. T. Speech Emotion Recognition Based on Speech Segment Using LSTM with Attention Model / B. T. Atmaja, M. Akagi. – Текст : непосредственный // IEEE International Conference on Signals and Systems (ICSigSys). – 2019. – P. 40–44. – DOI: 10.1109/ICSIGSYS.2019.8811080.
14. Li, D. Word-Level Deep Sign Language Recognition from Video: A New Large-Scale Dataset and Methods Comparison / D. Li, C. Rodriguez, X. Yu, H. Li. – Текст : непосредственный // Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV). – 2020. – P. 1459–1469. – DOI: 10.1109/WACV45572.2020.9093512.
15. Camgoz, N. C. BosphorusSign: A Turkish Sign Language Recognition Corpus in Health and Finance Domains / N. C. Camgoz, A. Kindiroglu, S. Karabuklu

[et al.]. – Текст : непосредственный // Proceedings of the International Conference on Language Resources and Evaluation (LREC). – 2016.

16. Min, Y. Visual Alignment Constraint for Continuous Sign Language Recognition / Y. Min, A. Hao, X. Chai, X. Chen. – Текст : непосредственный // Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). – 2021. – P. 11522–11531. – DOI: 10.1109/ICCV48922.2021.01134.

17. Ko, S.-K. Neural Sign Language Translation Based on Human Keypoint Estimation / S.-K. Ko, C. J. Kim, H. Jung, C. Cho. – Текст : непосредственный // Applied Sciences. – 2019. – Vol. 9, № 13. – P. 2683. – DOI: 10.3390/app9132683.

18. Kim, S. Challenges in Automatic Sign Language Recognition: A Systematic Review / S. Kim, J. Kim. – Текст : непосредственный // IEEE Access. – 2021. – Vol. 9. – P. 170106–170130.

19. Kapitanov, A. SLOVO: A Russian Sign Language Dataset / A. Kapitanov, K. Kvanchiani, A. Nagaev [et al.]. – Текст : электронный // arXiv. – 2023. – URL: <https://arxiv.org/abs/2305.14527> (дата обращения: 10.02.2026).

20. Carreira, J. Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset / J. Carreira, A. Zisserman. – Текст : непосредственный // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. – 2017. – P. 6299–6308.

21. Xie, S. Rethinking Spatiotemporal Feature Learning: Speed-Accuracy Trade-offs in Video Classification / S. Xie, C. Sun, J. Huang [et al.]. – Текст : непосредственный // Proceedings of the European Conference on Computer Vision. – 2018. – P. 305–321.

22. Feichtenhofer, C. SlowFast Networks for Video Recognition / C. Feichtenhofer, H. Fan, J. Malik, K. He. – Текст : непосредственный // Proceedings of the IEEE/CVF International Conference on Computer Vision. – 2019. – P. 6202–6211.

23. Yang, Z. Effective Whole-Body Pose Estimation with Two-Stages Distillation / Z. Yang, A. Zeng, C. Yuan, Y. Li. – Текст : электронный // Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops

(ICCVW). – 2023. – URL: <https://arxiv.org/abs/2307.15880> (дата обращения: 12.06.2026).

24. Ge, Z. YOLOX: Exceeding YOLO Series in 2021 / Z. Ge, S. Liu, F. Wang [et al.]. – Текст : электронный // arXiv. – 2021. – URL: <https://arxiv.org/abs/2107.08430> (дата обращения: 12.06.2026).

25. Jiang, T. RTMPose: Real-Time Multi-Person Pose Estimation Based on MMPose / T. Jiang, P. Lu, L. Zhang [et al.]. – Текст : электронный // arXiv. – 2023. – URL: <https://arxiv.org/abs/2303.07399> (дата обращения: 12.06.2026).

26. Yan, S. Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition / S. Yan, Y. Xiong, D. Lin. – Текст : непосредственный // Proceedings of the AAAI Conference on Artificial Intelligence. – 2018. – Vol. 32, № 1. – P. 7444–7452.

27. Shi, L. Skeleton-Based Action Recognition with Directed Graph Neural Networks / L. Shi, Y. Zhang, J. Cheng, H. Lu. – Текст : непосредственный // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. – 2019. – P. 7912–7921.

28. Новопольцев, М. С. Распознавание русского жестового языка: распознаём до 3+ жестов в секунду на обычном ПК без видеокарты / М. С. Новопольцев. – Текст : электронный // Хабр : блог компании Сбер. – 2023. – URL: <https://habr.com/ru/companies/sberbank/articles/775688/> (дата обращения: 15.03.2026).

29. Camgoz, N. C. Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation / N. C. Camgoz, O. Koller, S. Hadfield, R. Bowden. – Текст : непосредственный // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. – 2020. – P. 10023–10033.

30. Radford, A. Robust Speech Recognition via Large-Scale Weak Supervision / A. Radford, J. W. Kim, T. Xu [et al.]. – Текст : электронный // arXiv. – 2022. – URL: <https://arxiv.org/abs/2212.04356> (дата обращения: 10.02.2026).

31. Hu, E. LoRA: Low-Rank Adaptation of Large Language Models / E. Hu, Y. Shen, P. Wallis [et al.]. – Текст : электронный // arXiv. – 2021. – URL: <https://arxiv.org/abs/2106.09685> (дата обращения: 10.02.2026).
33. Dettmers, T. QLoRA: Efficient Finetuning of Quantized LLMs / T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. – Текст : электронный // arXiv. – 2023. – URL: <https://arxiv.org/abs/2305.14314> (дата обращения: 10.02.2026).
34. Qwen Team. Qwen2 Technical Report / Qwen Team. – Текст : электронный // arXiv. – 2024. – URL: <https://arxiv.org/abs/2407.10671> (дата обращения: 10.02.2026).
35. Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [et al.]. – Текст : непосредственный // Advances in Neural Information Processing Systems. – 2020. – Vol. 33. – P. 9459–9474.
36. Chen, X. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation / X. Chen, S. Zhang, P. Liu [et al.]. – Текст : электронный // arXiv. – 2024. – URL: <https://arxiv.org/abs/2309.07597> (дата обращения: 10.02.2026).
37. Saeki, T. UTMOS: UTokyo-SaruLab System for VoiceMOS Challenge 2022 / T. Saeki, D. Xin, W. Nakata [et al.]. – Текст : электронный // arXiv. – 2022. – URL: <https://arxiv.org/abs/2204.02152> (дата обращения: 10.02.2026).
38. Стерлинг, Г. Наш новый LLM-based синтез речи / Г. Стерлинг. – Текст : электронный // Хабр : блог компании Сбер. – 2024. – URL: <https://habr.com/ru/companies/sberbank/articles/966640/> (дата обращения: 15.03.2026).

ПРИЛОЖЕНИЕ А

Листинг А.1 – Загрузчик датасета SlovoDataset

```
class SlovoDataset:
    def __init__(self, root, split="all", exclude_no_event=True):
        self.root = Path(root)
        self.split = split
        self.exclude_no_event = exclude_no_event
        self._samples = []
        self._label_map = {}
        self._load()

    @staticmethod
    def _parse_split(row):
        if "split" in row and row["split"]:
            return row["split"]
        train_val = row.get("train", "True")
        return "train" if str(train_val).lower() in ("true", "1", "yes") else
"val"

    def _load(self):
        ann = self._find_annotations()
        classes, raw = [], []
        with open(ann, encoding="utf-8") as f:
            for row in csv.DictReader(f):
                text = row["text"]
                if self.exclude_no_event and text == "no_event":
                    continue
                s = self._parse_split(row)
                if self.split != "all" and s != self.split:
                    continue
                if text not in classes:
                    classes.append(text)
                raw.append(**row, "_split": s)
        classes.sort()
        self._label_map = {name: i for i, name in enumerate(classes)}
        for row in raw:
```

```

s = row["_split"]
self._samples.append(SlovoSample(
    video_path=self._find_video(row, s),
    label=self._label_map[row["text"]],
    label_name=row["text"],
    start_frame=int(row.get("begin", 0) or 0),
    end_frame=int(row.get("end", -1) or -1),
    split=s,
))

```

Листинг А.2 – Ключевые утилиты MLflow

```

def setup_mlflow(experiment_name=None):
    try:
        import mlflow

        tracking_uri = os.environ.get("MLFLOW_TRACKING_URI", "http://localhost:5000")

        mlflow.set_tracking_uri(tracking_uri)

        if experiment_name:
            mlflow.set_experiment(experiment_name)
    except ImportError:
        pass

@contextmanager
def mlflow_run(experiment_name, run_name, tags=None, nested=False):
    try:
        import mlflow

        tracking_uri = os.environ.get("MLFLOW_TRACKING_URI", "http://localhost:5000")

        mlflow.set_tracking_uri(tracking_uri)

        exp_id = get_or_create_experiment(experiment_name)

        with mlflow.start_run(experiment_id=exp_id, run_name=run_name,
                               tags=tags or {}, nested=nested) as run:

            yield run
    except ImportError:
        class _Noop:
            info = type("I", (), {"run_id": "local"})()

        yield _Noop()

```

Листинг А.3 – Нормализация ключевых точек

```
def hip_centered_normalize(keypoints: np.ndarray) -> np.ndarray:
    # keypoints: (T, 75, 3) – pose(33) + lhand(21) + rhand(21)
    hip_center = (keypoints[:, 11, :] + keypoints[:, 12, :]) / 2.0
    kp = keypoints - hip_center[:, np.newaxis, :]
    shoulder_width = np.linalg.norm(
        keypoints[:, 5, :2] - keypoints[:, 6, :2], axis=-1
    )
    valid = shoulder_width > 1e-6
    scale = np.where(valid, shoulder_width, 1.0)
    kp[valid] = kp[valid] / scale[valid, np.newaxis, np.newaxis]
    return kp.astype(np.float32)
```